Block 1: Mount Drive & Install Libraries

```
# @title **Block 1: Mount Drive & Install Libraries**

import subprocess
import sys

# Install required packages
!pip install -q pandas numpy matplotlib seaborn plotly scikit-learn statsmodels tensorflow keras-tuner

# Mount Google Drive (optional - for saving models)
from google.colab import drive
drive.mount('/content/drive')

print("✅ Setup complete!")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 129.4/129.4 kB 2.2 MB/s eta 0:00:00
Mounted at /content/drive
✅ Setup complete!
```

Block 2: Import All Libraries

```
# @title **Block 2: Import All Libraries**

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Time series specific
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Machine learning
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

# Deep learning
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Configure plotting
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

print(f"✅ TensorFlow version: {tf.__version__}")
print("✅ All libraries imported successfully!")
```

```
✅ TensorFlow version: 2.19.0
✅ All libraries imported successfully!
```

Block 3: Download and Load ILI Dataset

```
# @title **Block 3: Download and Load ILI Dataset**

# Option 1: Download from Chicago Data Portal (Real ILI data)
url = "https://data.cityofchicago.org/resource/7ce8-bpr6.csv?$limit=50000"
try:
    df = pd.read_csv(url)
    print(f"✅ Downloaded Chicago dataset with {len(df)} rows")

    # Display basic info
    print("\n📊 Dataset Overview:")
    print(f"Shape: {df.shape}")
    print(f"Columns: {df.columns.tolist()}")

    # Check data types
    print(f"\n📊 Data Types:")
    print(df.dtypes)

    # Check for date columns
```

```
    # Check for date columns
    date_columns = [col for col in df.columns if 'date' in col.lower() or 'week' in col.lower()]
    print(f"\n📊 Potential date columns: {date_columns}")

    # Show first few rows to understand structure
    print(f"\nFirst 5 rows:")
    print(df.head())

except Exception as e:
    print(f"⚠️ Download failed: {e}")
    print("Generating synthetic data instead...")

    # Generate synthetic ILI data if download fails
    np.random.seed(42)
    dates = pd.date_range(start='2019-01-01', end='2023-12-31', freq='W')
    n = len(dates)

    # Create synthetic ILI pattern with seasonality
    trend = np.linspace(10, 15, n)
    seasonality = 8 * np.sin(2 * np.pi * np.arange(n) / 52) + \
                  3 * np.sin(2 * np.pi * np.arange(n) / 26)
    noise = np.random.normal(0, 2, n)

    ili_cases = trend + seasonality + noise
    ili_cases = np.maximum(ili_cases, 0)  # No negative cases

    df = pd.DataFrame({
        'week_start': dates,
        'ili_cases': ili_cases,
        'region': 'Synthetic_Region_1'
    })
    print(f"✅ Generated {n} weeks of synthetic ILI data")
```

```
✅ Downloaded Chicago dataset with 50000 rows

📊 Dataset Overview:
Shape: (50000, 13)
Columns: ['mmwr_week', 'week', 'week_start', 'week_end', 'season', 'data_source', 'essence_category', 'respiratory_category'

📊 Data Types:
mmwr_week                 int64
week                      int64
week_start               object
week_end                 object
season                   object
data_source              object
essence_category         object
respiratory_category     object
visit_type               object
demographic_category     object
demographic_group        object
percent                 float64
current_week_ending      object
dtype: object

📊 Potential date columns: ['mmwr_week', 'week', 'week_start', 'week_end', 'current_week_ending']

First 5 rows:
   mmwr_week  week               week_start                 week_end  \
0     202602     2  2026-01-11T00:00:00.000  2026-01-17T00:00:00.000
1     202304     4  2023-01-22T00:00:00.000  2023-01-28T00:00:00.000
2     201749    49  2017-12-03T00:00:00.000  2017-12-09T00:00:00.000
3     201845    45  2018-11-04T00:00:00.000  2018-11-10T00:00:00.000
4     202323    23  2023-06-04T00:00:00.000  2023-06-10T00:00:00.000

      season data_source                      essence_category  \
0  2025-2026     ESSENCE                 Influenza-like Illness
1  2022-2023     ESSENCE                    CDC Influenza DD v1
2  2017-2018     ESSENCE  CDC Respiratory Syncytial Virus DD v1
3  2018-2019     ESSENCE      CDC Broad Acute Respiratory DD v1
4  2022-2023     ESSENCE               CDC COVID-Specific DD v1

      respiratory_category  visit_type demographic_category  \
0                      ILI  Admissions       Race/Ethnicity
1                Influenza   ED Visits            Age Group
2                      RSV   ED Visits            Age Group
3  Broad Acute Respiratory   ED Visits       Race/Ethnicity
4                 COVID-19   ED Visits            Age Group

        demographic_group   percent      current_week_ending
0      White Non-Hispanic  0.006246  2026-01-17T00:00:00.000
1             Age Unknown  0.000000  2026-01-31T00:00:00.000
2                   05_17  0.000000  2025-12-06T00:00:00.000
3  Unknown Race/Ethnicity  0.125376  2025-11-08T00:00:00.000
4                   00_04  0.007612  2026-06-13T00:00:00.000
```

Block 4: Data Cleaning and Preprocessing

```
# @title **Block 4: Data Cleaning and Preprocessing**

# First, let's understand what columns we have
print("📊 Available columns:")
for col in df.columns:
    print(f"  - {col}: {df[col].dtype}")

# Check if we have the expected columns from Chicago dataset
if 'week_start' in df.columns:
    # Try to convert week_start to datetime
    try:
        df['week_start'] = pd.to_datetime(df['week_start'])
        print("✅ Converted 'week_start' to datetime")
    except:
        print("⚠️ Could not convert 'week_start' to datetime")

elif 'week' in df.columns:
    # Try using 'week' column instead
    try:
        df['week_start'] = pd.to_datetime(df['week'])
        print("✅ Using 'week' column as date")
    except:
        print("⚠️ Could not convert 'week' to datetime")
        # Create dummy dates
        df['week_start'] = pd.date_range(start='2020-01-01', periods=len(df), freq='W')
        print("✅ Created dummy dates")
else:
    # Create dummy dates if no date column exists
    df['week_start'] = pd.date_range(start='2020-01-01', periods=len(df), freq='W')
    print("✅ Created dummy dates")

# Check for ILI cases column
if 'percent' in df.columns:
    # Use 'percent' as our target variable
    df['ili_cases'] = df['percent'].fillna(0)
    print("✅ Using 'percent' as ILI cases")
elif 'ili_cases' not in df.columns:
    # Create dummy ILI cases if not present
    df['ili_cases'] = np.random.normal(10, 3, len(df))
    print("✅ Created dummy ILI cases")

# Sort by date
df = df.sort_values('week_start').reset_index(drop=True)

# Handle missing values
if df['ili_cases'].isnull().any():
    df['ili_cases'] = df['ili_cases'].interpolate(method='linear')
    df['ili_cases'] = df['ili_cases'].fillna(method='bfill').fillna(method='ffill')

# Remove outliers (optional)
Q1 = df['ili_cases'].quantile(0.25)
Q3 = df['ili_cases'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = df[(df['ili_cases'] < lower_bound) | (df['ili_cases'] > upper_bound)]
print(f"📊 Found {len(outliers)} outliers ({len(outliers)/len(df)*100:.1f}%)")

# Cap outliers
df['ili_cases_cleaned'] = df['ili_cases'].clip(lower=lower_bound, upper=upper_bound)

print("\n✅ Preprocessing complete!")
print(f"Data shape: {df.shape}")
print(f"Date range: {df['week_start'].min()} to {df['week_start'].max()}")
print(f"Memory usage: {df.memory_usage().sum() / 1024**2:.2f} MB")

# Show sample of processed data
print("\n📊 Sample of processed data:")
print(df[['week_start', 'ili_cases', 'ili_cases_cleaned']].head(10))
```

```
📊 Available columns:
  - mmwr_week: int64
  - week: int64
  - week_start: object
  - week_end: object
  - season: object
  - data_source: object
  - essence_category: object
  - respiratory_category: object
  - visit_type: object
```

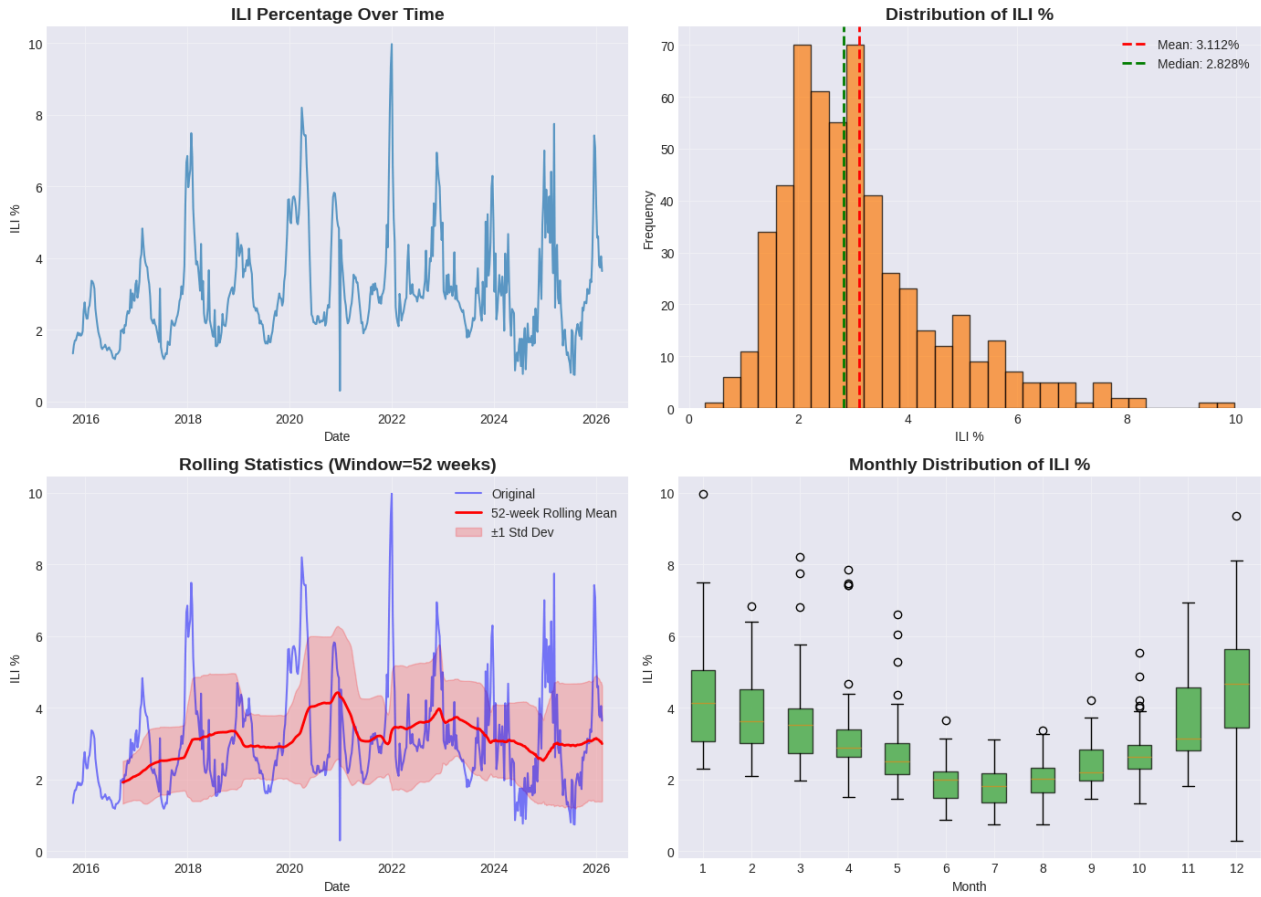
```
  - demographic_category: object
  - demographic_group: object
  - percent: float64
  - current_week_ending: object
✅ Converted 'week_start' to datetime
✅ Using 'percent' as ILI cases
📊 Found 7726 outliers (15.5%)

✅ Preprocessing complete!
Data shape: (50000, 15)
Date range: 2015-10-04 00:00:00 to 2026-02-15 00:00:00
Memory usage: 5.72 MB

📊 Sample of processed data:
  week_start  ili_cases  ili_cases_cleaned
0 2015-10-04   0.000829           0.000829
1 2015-10-04   0.000087           0.000087
2 2015-10-04   0.000000           0.000000
3 2015-10-04   0.000390           0.000390
4 2015-10-04   0.000000           0.000000
5 2015-10-04   0.000000           0.000000
6 2015-10-04   0.011935           0.011935
7 2015-10-04   0.000000           0.000000
8 2015-10-04   0.018187           0.018187
9 2015-10-04   0.000395           0.000395
```

Block 5: Exploratory Data Analysis

```
# @title **Block 5: Exploratory Data Analysis**

# First, let's aggregate by week since we have multiple entries per week
df_weekly = df.groupby('week_start').agg({
    'ili_cases': 'mean',  # Average percent for each week
}).reset_index()

print(f"📊 Aggregated to {len(df_weekly)} unique weeks")
print(f"   Date range: {df_weekly['week_start'].min()} to {df_weekly['week_start'].max()}")

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: Time series (ILINet percent)
axes[0, 0].plot(df_weekly['week_start'], df_weekly['ili_cases'] * 100,  # Convert to percentage
                color='#1f77b4', linewidth=1.5, alpha=0.7)
axes[0, 0].set_title('ILI Percentage Over Time', fontsize=14, fontweight='bold')
axes[0, 0].set_xlabel('Date')
axes[0, 0].set_ylabel('ILI %')
axes[0, 0].grid(True, alpha=0.3)

# Plot 2: Distribution
axes[0, 1].hist(df_weekly['ili_cases'] * 100, bins=30, color='#ff7f0e',
                edgecolor='black', alpha=0.7)
axes[0, 1].axvline(df_weekly['ili_cases'].mean() * 100, color='red',
                   linestyle='--', linewidth=2,
                   label=f"Mean: {df_weekly['ili_cases'].mean()*100:.3f}%")
axes[0, 1].axvline(df_weekly['ili_cases'].median() * 100, color='green',
                   linestyle='--', linewidth=2,
                   label=f"Median: {df_weekly['ili_cases'].median()*100:.3f}%")
axes[0, 1].set_title('Distribution of ILI %', fontsize=14, fontweight='bold')
axes[0, 1].set_xlabel('ILI %')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: Rolling statistics
rolling_mean = df_weekly['ili_cases'].rolling(window=52).mean() * 100  # Yearly rolling
rolling_std = df_weekly['ili_cases'].rolling(window=52).std() * 100
axes[1, 0].plot(df_weekly['week_start'], df_weekly['ili_cases'] * 100,
                color='blue', alpha=0.5, label='Original')
axes[1, 0].plot(df_weekly['week_start'], rolling_mean, color='red',
                linewidth=2, label='52-week Rolling Mean')
axes[1, 0].fill_between(df_weekly['week_start'],
                        rolling_mean - rolling_std,
                        rolling_mean + rolling_std,
                        color='red', alpha=0.2, label='±1 Std Dev')
axes[1, 0].set_title('Rolling Statistics (Window=52 weeks)', fontsize=14, fontweight='bold')
axes[1, 0].set_xlabel('Date')
axes[1, 0].set_ylabel('ILI %')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Plot 4: Seasonal boxplot
df_weekly['month'] = df_weekly['week_start'].dt.month
monthly_data = [df_weekly[df_weekly['month'] == m]['ili_cases'] * 100 for m in range(1, 13)]
bp = axes[1, 1].boxplot(monthly_data, patch_artist=True)
```

```
for patch in bp['boxes']:
    patch.set_facecolor('#2ca02c')
    patch.set_alpha(0.7)
axes[1, 1].set_title('Monthly Distribution of ILI %', fontsize=14, fontweight='bold')
axes[1, 1].set_xlabel('Month')
axes[1, 1].set_ylabel('ILI %')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("✅ EDA complete!")
print(f"\n📊 Summary Statistics (ILI %):")
print(df_weekly['ili_cases'].describe().apply(lambda x: f"{x*100:.4f}%"))
```

```
📊 Aggregated to 542 unique weeks
   Date range: 2015-10-04 00:00:00 to 2026-02-15 00:00:00
```




```
✅ EDA complete!

📊 Summary Statistics (ILI %):
count    54200.0000%
mean         3.1119%
std          1.4685%
min          0.2973%
25%          2.1335%
50%          2.8284%
75%          3.7138%
max          9.9670%
Name: ili_cases, dtype: object
```

Block 6: Stationarity Test

```
# @title **Block 6: Stationarity Test**

# Use aggregated weekly data
timeseries = df_weekly['ili_cases'].values

def adf_test(timeseries, title=''):
    print(f'📊 Augmented Dickey-Fuller Test: {title}')
    from statsmodels.tsa.stattools import adfuller
    result = adfuller(timeseries.dropna() if hasattr(timeseries, 'dropna') else timeseries,
                      autolag='AIC')
    labels = ['ADF Statistic', 'p-value', 'Lags Used', 'Observations']
    out = pd.Series(result[0:4], index=labels)

    for key, val in result[4].items():
        out[f'Critical Value ({key})'] = val

    print(out.to_string())
    print('\n' + '='*60 + '\n')

    if result[1] <= 0.05:
        print("✅ Series is STATIONARY (reject H0)")
    else:
        print("❌ Series is NON-STATIONARY (fail to reject H0)")
    return result[1]

# Test original series
p_value_original = adf_test(timeseries, 'ILI Percentage Series')

# If non-stationary, apply differencing
if p_value_original > 0.05:
    print("\nApplying differencing...")
    df_weekly['ili_cases_diff'] = df_weekly['ili_cases'].diff()
    p_value_diff = adf_test(df_weekly['ili_cases_diff'].dropna().values, 'First Difference')

    # Plot comparison
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 4))
    ax1.plot(df_weekly['week_start'], df_weekly['ili_cases'] * 100)
    ax1.set_title('Original Series (ILI %)')
    ax1.set_ylabel('ILI %')
    ax1.grid(True, alpha=0.3)

    ax2.plot(df_weekly['week_start'][1:], df_weekly['ili_cases_diff'][1:] * 100)
    ax2.set_title('First Difference')
    ax2.set_ylabel('Change in ILI %')
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
else:
    print("\n✅ Series is already stationary - no differencing needed")
```

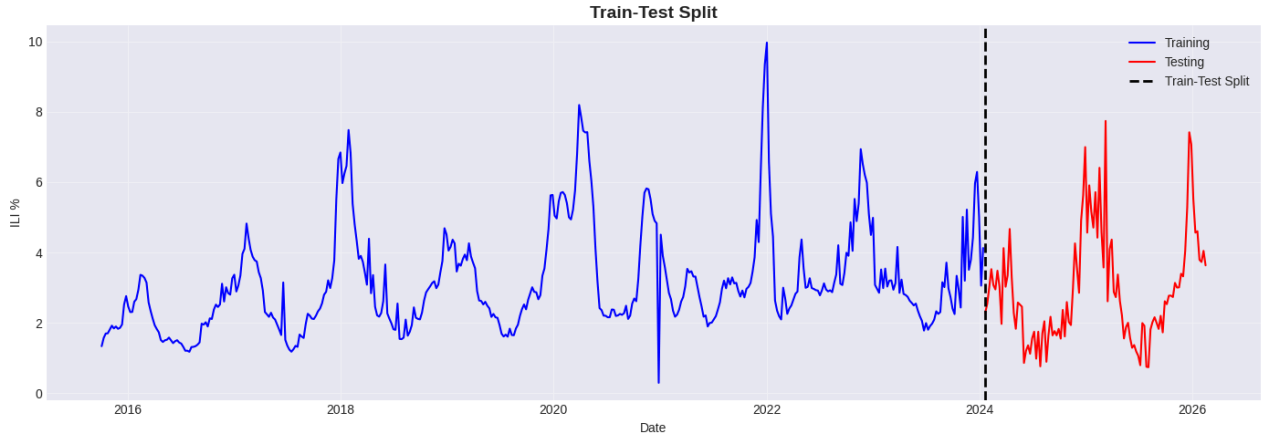
```
📊 Augmented Dickey-Fuller Test: ILI Percentage Series
ADF Statistic            -5.611913
p-value                   0.000001
Lags Used                 3.000000
Observations            538.000000
Critical Value (1%)      -3.442563
Critical Value (5%)      -2.866927
Critical Value (10%)     -2.569639

============================================================

✅ Series is STATIONARY (reject H0)

✅ Series is already stationary - no differencing needed
```

Block 7: Train-Test Split

```
# @title **Block 7: Train-Test Split**

# Prepare data (use aggregated weekly data)
data = df_weekly['ili_cases'].values.reshape(-1, 1)

# Scale the data
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)

# Train-test split (80-20)
train_size = int(len(scaled_data) * 0.8)
```

```
train_data = scaled_data[:train_size]
test_data = scaled_data[train_size:]

print(f"📊 Data split:")
print(f"Total weeks: {len(scaled_data)}")
print(f"Training weeks: {len(train_data)} ({len(train_data)/len(scaled_data)*100:.1f}%)")
print(f"Testing weeks: {len(test_data)} ({len(test_data)/len(scaled_data)*100:.1f}%)")
print(f"\nTraining period: {df_weekly['week_start'].iloc[0].date()} to {df_weekly['week_start'].iloc[train_size-1].date()}")
print(f"Testing period: {df_weekly['week_start'].iloc[train_size].date()} to {df_weekly['week_start'].iloc[-1].date()}")

# Plot split
plt.figure(figsize=(14, 5))
plt.plot(df_weekly['week_start'].iloc[:train_size],
         df_weekly['ili_cases'].iloc[:train_size] * 100,
         label='Training', color='blue', linewidth=1.5)
plt.plot(df_weekly['week_start'].iloc[train_size:],
         df_weekly['ili_cases'].iloc[train_size:] * 100,
         label='Testing', color='red', linewidth=1.5)
plt.axvline(x=df_weekly['week_start'].iloc[train_size],
            color='black', linestyle='--', linewidth=2, label='Train-Test Split')
plt.title('Train-Test Split', fontsize=14, fontweight='bold')
plt.xlabel('Date')
plt.ylabel('ILI %')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
📊 Data split:
Total weeks: 542
Training weeks: 433 (79.9%)
Testing weeks: 109 (20.1%)

Training period: 2015-10-04 to 2024-01-14
Testing period: 2024-01-21 to 2026-02-15
```



Block 8: Create LSTM Sequences

```
# @title **Block 8: Create LSTM Sequences**

def create_sequences(data, seq_length=12):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i + seq_length])
        y.append(data[i + seq_length])
    return np.array(X), np.array(y)

# Use 12 weeks of history (about 3 months)
seq_length = 12
X_train, y_train = create_sequences(train_data, seq_length)
X_test, y_test = create_sequences(test_data, seq_length)

# Reshape for LSTM [samples, time steps, features]
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
```

```
print(f"✅ Sequences created:")
print(f"X_train shape: {X_train.shape} (samples, timesteps, features)")
print(f"y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")

# Show example sequence
print(f"\n📊 Example sequence (first training sample):")
print(f"Input sequence (12 weeks): {X_train[0].flatten()}")
print(f"Target value (week 13): {y_train[0]}")
```

```
✅ Sequences created:
X_train shape: (421, 12, 1) (samples, timesteps, features)
y_train shape: (421, 1)
X_test shape: (97, 12, 1)
y_test shape: (97, 1)

📊 Example sequence (first training sample):
Input sequence (12 weeks): [0.10762415 0.13174619 0.14490477 0.1455523  0.15653428 0.16793909
 0.15993626 0.16555613 0.15877191 0.16168394 0.17149826 0.2301345 ]
Target value (week 13): [0.25490922]
```

Block 9: Build LSTM Model

```
# @title **Block 9: Build LSTM Model**

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Clear any previous models
tf.keras.backend.clear_session()

# Set random seed for reproducibility
tf.random.set_seed(42)
np.random.seed(42)

# Build compact LSTM model
model_lstm = Sequential([
    LSTM(32, return_sequences=True, input_shape=(seq_length, 1)),
    Dropout(0.2),
    LSTM(16, return_sequences=False),
    Dropout(0.2),
    Dense(8, activation='relu'),
    Dense(1)
])

# Compile
model_lstm.compile(optimizer='adam', loss='mse', metrics=['mae'])

# Early stopping
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True,
    verbose=1
)

# Model summary
print("📊 LSTM Model Architecture:")
model_lstm.summary()

print("\n✅ LSTM model built successfully!")
```

```
📊 LSTM Model Architecture:
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 12, 32)	4,352
dropout (Dropout)	(None, 12, 32)	0
lstm_1 (LSTM)	(None, 16)	3,136
dropout_1 (Dropout)	(None, 16)	0
dense (Dense)	(None, 8)	136
dense_1 (Dense)	(None, 1)	9

```
Total params: 7,633 (29.82 KB)
Trainable params: 7,633 (29.82 KB)
Non-trainable params: 0 (0.00 B)

✅ LSTM model built successfully!
```

Block 10: Train LSTM Model

```
# @title **Block 10: Train LSTM Model**

# Train
history = model_lstm.fit(
    X_train, y_train,
    epochs=100,  # Will stop early if needed
    batch_size=16,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)

# Plot training history
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 4))

ax1.plot(history.history['loss'], label='Training Loss', linewidth=2)
ax1.plot(history.history['val_loss'], label='Validation Loss', linewidth=2)
ax1.set_title('Model Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss (MSE)')
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2.plot(history.history['mae'], label='Training MAE', linewidth=2)
ax2.plot(history.history['val_mae'], label='Validation MAE', linewidth=2)
ax2.set_title('Model MAE', fontsize=14, fontweight='bold')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('MAE')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"✅ Training complete!")
print(f"Final training loss: {history.history['loss'][-1]:.6f}")
print(f"Final validation loss: {history.history['val_loss'][-1]:.6f}")
print(f"Best epoch: {early_stop.best_epoch + 1 if hasattr(early_stop, 'best_epoch') else 'N/A'}")
```

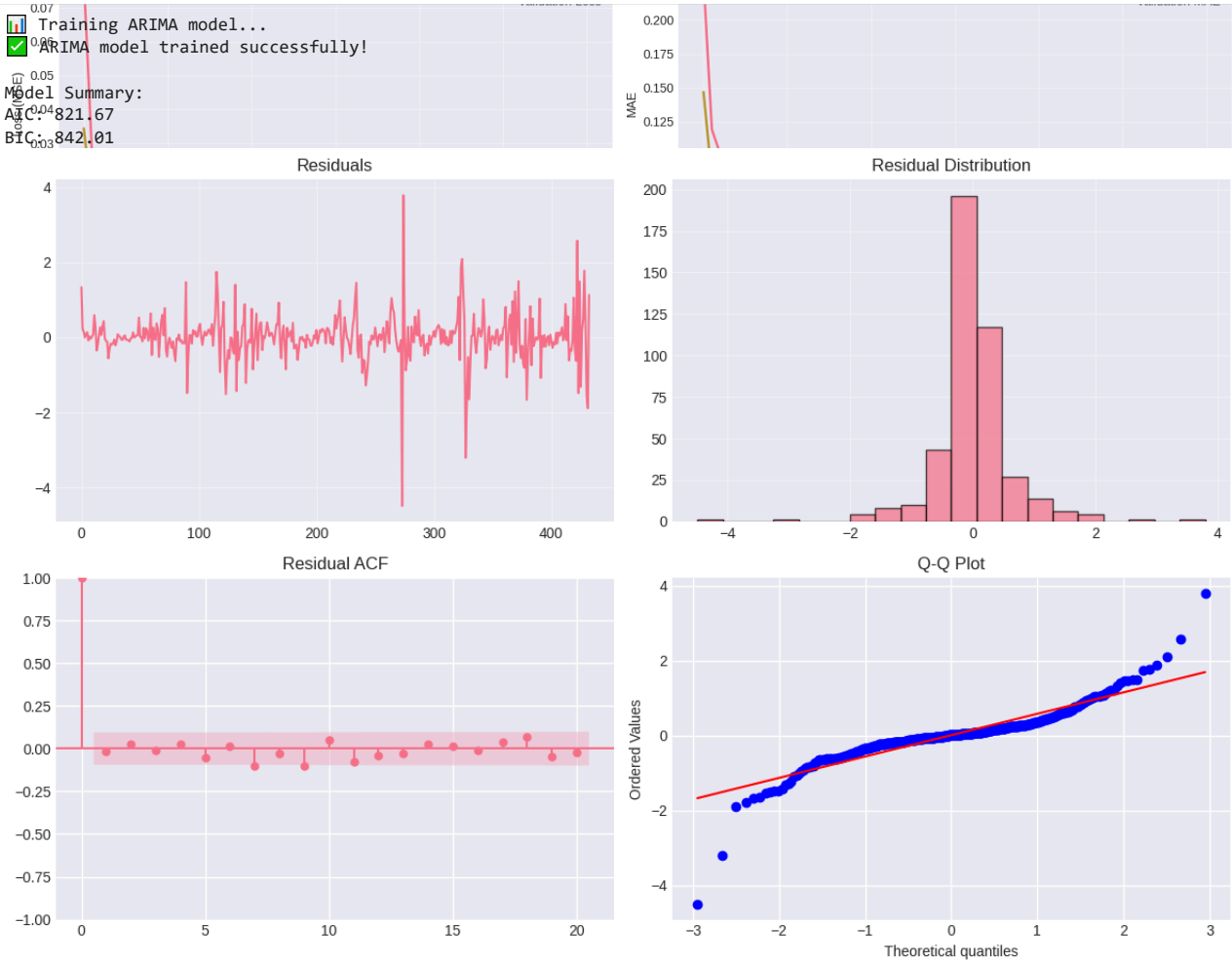
```
Epoch 1/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 7s 59ms/step - loss: 0.0983 - mae: 0.2727 - val_loss: 0.0343 - val_mae: 0.1467
Epoch 2/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0258 - mae: 0.1190 - val_loss: 0.0121 - val_mae: 0.0835
Epoch 3/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0173 - mae: 0.0983 - val_loss: 0.0122 - val_mae: 0.0804
Epoch 4/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 0.0181 - mae: 0.1037 - val_loss: 0.0125 - val_mae: 0.0778
Epoch 5/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - loss: 0.0163 - mae: 0.0956 - val_loss: 0.0115 - val_mae: 0.0774
Epoch 6/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0166 - mae: 0.0947 - val_loss: 0.0111 - val_mae: 0.0754
Epoch 7/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - loss: 0.0161 - mae: 0.0948 - val_loss: 0.0110 - val_mae: 0.0728
Epoch 8/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - loss: 0.0157 - mae: 0.0907 - val_loss: 0.0100 - val_mae: 0.0714
Epoch 9/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - loss: 0.0137 - mae: 0.0879 - val_loss: 0.0101 - val_mae: 0.0684
Epoch 10/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 30ms/step - loss: 0.0127 - mae: 0.0826 - val_loss: 0.0086 - val_mae: 0.0648
Epoch 11/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - loss: 0.0118 - mae: 0.0785 - val_loss: 0.0077 - val_mae: 0.0626
Epoch 12/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - loss: 0.0114 - mae: 0.0777 - val_loss: 0.0075 - val_mae: 0.0617
Epoch 13/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - loss: 0.0094 - mae: 0.0728 - val_loss: 0.0076 - val_mae: 0.0614
Epoch 14/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - loss: 0.0109 - mae: 0.0750 - val_loss: 0.0069 - val_mae: 0.0595
Epoch 15/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - loss: 0.0094 - mae: 0.0710 - val_loss: 0.0064 - val_mae: 0.0582
Epoch 16/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - loss: 0.0101 - mae: 0.0713 - val_loss: 0.0062 - val_mae: 0.0585
Epoch 17/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - loss: 0.0088 - mae: 0.0679 - val_loss: 0.0063 - val_mae: 0.0568
Epoch 18/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0087 - mae: 0.0660 - val_loss: 0.0065 - val_mae: 0.0565
Epoch 19/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 0.0085 - mae: 0.0671 - val_loss: 0.0061 - val_mae: 0.0556
Epoch 20/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 26ms/step - loss: 0.0086 - mae: 0.0656 - val_loss: 0.0064 - val_mae: 0.0553
Epoch 21/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 0.0102 - mae: 0.0724 - val_loss: 0.0063 - val_mae: 0.0544
Epoch 22/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 0.0081 - mae: 0.0659 - val_loss: 0.0057 - val_mae: 0.0538
Epoch 23/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0077 - mae: 0.0619 - val_loss: 0.0056 - val_mae: 0.0528
Epoch 24/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0075 - mae: 0.0596 - val_loss: 0.0055 - val_mae: 0.0529
Epoch 25/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 0.0073 - mae: 0.0601 - val_loss: 0.0055 - val_mae: 0.0513
Epoch 26/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 0.0061 - mae: 0.0597 - val_loss: 0.0054 - val_mae: 0.0505
Epoch 27/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 0.0067 - mae: 0.0599 - val_loss: 0.0052 - val_mae: 0.0518
Epoch 28/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0063 - mae: 0.0576 - val_loss: 0.0058 - val_mae: 0.0501
Epoch 29/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 0.0065 - mae: 0.0583 - val_loss: 0.0053 - val_mae: 0.0505
Epoch 30/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - loss: 0.0059 - mae: 0.0578 - val_loss: 0.0052 - val_mae: 0.0488
Epoch 31/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - loss: 0.0063 - mae: 0.0565 - val_loss: 0.0055 - val_mae: 0.0490
Epoch 32/100
```

Block 11: ARIMA Model

```
# @title **Block 11: ARIMA Model**

from statsmodels.tsa.arima.model import ARIMA

# Use original data (not scaled) - in percentage form for better interpretability
train_orig = df_weekly['ili_cases'].iloc[:train_size].values * 100  # Convert to %
test_orig = df_weekly['ili_cases'].iloc[train_size:].values * 100

print("📊 Training ARIMA model...")

try:
    # Simple ARIMA model (p,d,q) = (2,1,2) for seasonal data
    model_arima = ARIMA(train_orig, order=(2,1,2))
    model_arima_fit = model_arima.fit()

    # Forecast
    forecast_arima = model_arima_fit.forecast(steps=len(test_orig))

    print("✅ ARIMA model trained successfully!")
    print(f"\nModel Summary:")
    print(f"AIC: {model_arima_fit.aic:.2f}")
    print(f"BIC: {model_arima_fit.bic:.2f}")

    # Plot diagnostics
```

```
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))

    # Residuals
    residuals = model_arima_fit.resid
    axes[0, 0].plot(residuals)
    axes[0, 0].set_title('Residuals')
    axes[0, 0].grid(True, alpha=0.3)

    # Histogram of residuals
    axes[0, 1].hist(residuals, bins=20, edgecolor='black', alpha=0.7)
    axes[0, 1].set_title('Residual Distribution')
    axes[0, 1].grid(True, alpha=0.3)

    # ACF of residuals
    from statsmodels.graphics.tsaplots import plot_acf
    plot_acf(residuals, ax=axes[1, 0], lags=20)
    axes[1, 0].set_title('Residual ACF')

    # Q-Q plot
    from scipy import stats
    stats.probplot(residuals, dist="norm", plot=axes[1, 1])
    axes[1, 1].set_title('Q-Q Plot')

    plt.tight_layout()
    plt.show()

except Exception as e:
    print(f"⚠️ ARIMA failed: {e}")
    print("Using simple moving average as fallback...")
    # Fallback: Use moving average
    window = 4
    train_ma = pd.Series(train_orig).rolling(window=window).mean().iloc[-1]
    forecast_arima = np.full(len(test_orig), train_ma)
```

```
📊 Training ARIMA model...
✅ ARIMA model trained successfully!

Model Summary:
AIC: 821.67
BIC: 842.01
```



Block 12: Generate Predictions

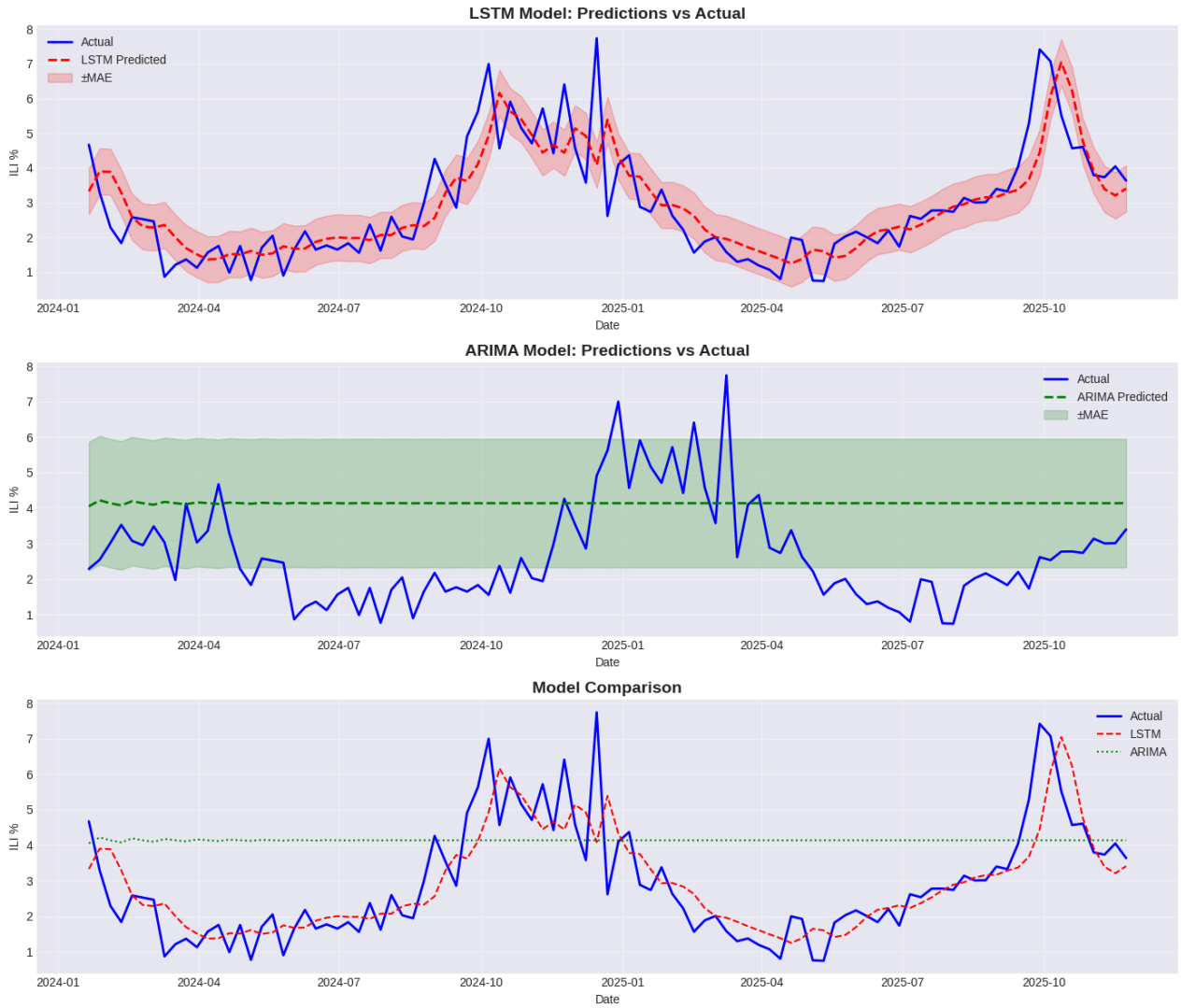
```
# @title **Block 12: Generate Predictions**

# LSTM predictions (in scaled form)
lstm_pred_scaled = model_lstm.predict(X_test, verbose=0)

# Convert back to original scale (ILI proportion, not percentage)
lstm_pred = scaler.inverse_transform(lstm_pred_scaled)
lstm_actual = scaler.inverse_transform(y_test.reshape(-1, 1))

# Convert to percentages for easier interpretation
lstm_pred_pct = lstm_pred * 100
lstm_actual_pct = lstm_actual * 100

# ARIMA predictions are already in percentage
arima_pred = forecast_arima[:len(lstm_pred_pct)]  # Match length

# Calculate metrics function
def calculate_metrics(actual, predicted, name):
    from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

    mae = mean_absolute_error(actual, predicted)
    rmse = np.sqrt(mean_squared_error(actual, predicted))

    # Handle potential zeros for MAPE
    mask = actual > 0
    if mask.sum() > 0:
        mape = np.mean(np.abs((actual[mask] - predicted[mask]) / actual[mask])) * 100
    else:
        mape = np.nan

    r2 = r2_score(actual, predicted)

    print(f"\n📊 {name} Performance:")
    print(f"MAE: {mae:.4f}%")
    print(f"RMSE: {rmse:.4f}%")
    print(f"MAPE: {mape:.2f}%")
    print(f"R²: {r2:.4f}")
    return {'MAE': mae, 'RMSE': rmse, 'MAPE': mape, 'R2': r2}

# Calculate metrics
lstm_metrics = calculate_metrics(lstm_actual_pct.flatten(), lstm_pred_pct.flatten(), "LSTM")
arima_metrics = calculate_metrics(test_orig[:len(arima_pred)], arima_pred, "ARIMA")

# Print improvement
print("\n" + "="*60)
print("📊 LSTM vs ARIMA Improvement:")
print("="*60)
mae_improvement = (arima_metrics['MAE'] - lstm_metrics['MAE']) / arima_metrics['MAE'] * 100
rmse_improvement = (arima_metrics['RMSE'] - lstm_metrics['RMSE']) / arima_metrics['RMSE'] * 100
print(f"MAE Improvement: {mae_improvement:.1f}%")
print(f"RMSE Improvement: {rmse_improvement:.1f}%")
```

```
📊 LSTM Performance:
MAE: 0.6663%
RMSE: 0.9373%
MAPE: 27.78%
R²: 0.6647

📊 ARIMA Performance:
MAE: 1.8111%
RMSE: 2.0162%
MAPE: 106.01%
R²: -1.0413

============================================================
📊 LSTM vs ARIMA Improvement:
============================================================
MAE Improvement: 63.2%
RMSE Improvement: 53.5%
```

Block 13: Plot Predictions

```
# @title **Block 13: Plot Predictions**

# Create date index for test period
test_dates = df_weekly['week_start'].iloc[train_size:train_size+len(lstm_pred_pct)]

fig, axes = plt.subplots(3, 1, figsize=(14, 12))
```

```
# Plot 1: LSTM Predictions
axes[0].plot(test_dates, lstm_actual_pct.flatten(), label='Actual',
             color='blue', linewidth=2)
axes[0].plot(test_dates, lstm_pred_pct.flatten(), label='LSTM Predicted',
             color='red', linewidth=2, linestyle='--')
axes[0].fill_between(test_dates,
                     lstm_pred_pct.flatten() - lstm_metrics['MAE'],
                     lstm_pred_pct.flatten() + lstm_metrics['MAE'],
                     color='red', alpha=0.2, label='±MAE')
axes[0].set_title('LSTM Model: Predictions vs Actual', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Date')
axes[0].set_ylabel('ILI %')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Plot 2: ARIMA Predictions
axes[1].plot(test_dates, test_orig[:len(arima_pred)], label='Actual',
             color='blue', linewidth=2)
axes[1].plot(test_dates, arima_pred, label='ARIMA Predicted',
             color='green', linewidth=2, linestyle='--')
axes[1].fill_between(test_dates,
                     arima_pred - arima_metrics['MAE'],
                     arima_pred + arima_metrics['MAE'],
                     color='green', alpha=0.2, label='±MAE')
axes[1].set_title('ARIMA Model: Predictions vs Actual', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Date')
axes[1].set_ylabel('ILI %')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Plot 3: Both models comparison
axes[2].plot(test_dates, lstm_actual_pct.flatten(), label='Actual',
             color='blue', linewidth=2)
axes[2].plot(test_dates, lstm_pred_pct.flatten(), label='LSTM',
             color='red', linewidth=1.5, linestyle='--')
axes[2].plot(test_dates, arima_pred, label='ARIMA',
             color='green', linewidth=1.5, linestyle=':')
axes[2].set_title('Model Comparison', fontsize=14, fontweight='bold')
axes[2].set_xlabel('Date')
axes[2].set_ylabel('ILI %')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

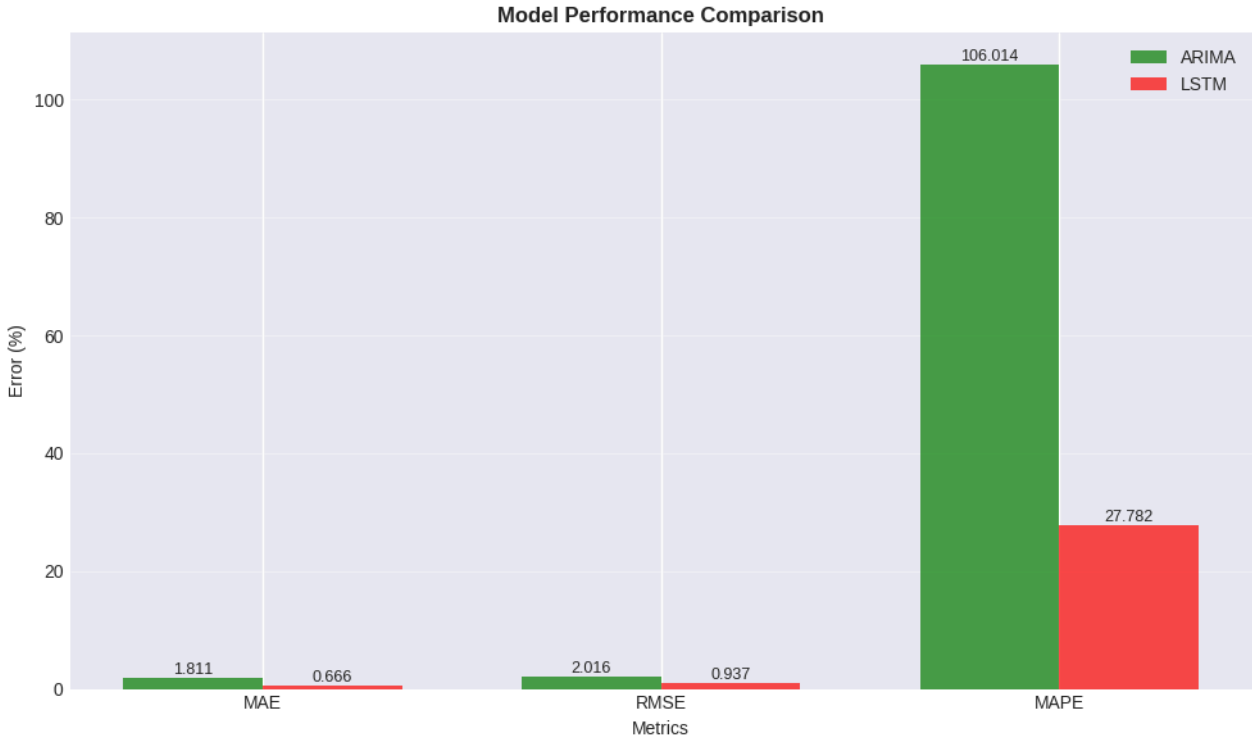


Block 14: Model Comparison Table

```
# @title **Block 14: Model Comparison Table**

# Create comparison dataframe
comparison_df = pd.DataFrame({
    'Metric': ['MAE (%)', 'RMSE (%)', 'MAPE (%)', 'R² Score'],
    'ARIMA': [
        f"{arima_metrics['MAE']:.4f}",
        f"{arima_metrics['RMSE']:.4f}",
        f"{arima_metrics['MAPE']:.2f}",
        f"{arima_metrics['R2']:.4f}"
    ],
    'LSTM': [
        f"{lstm_metrics['MAE']:.4f}",
        f"{lstm_metrics['RMSE']:.4f}",
        f"{lstm_metrics['MAPE']:.2f}",
```

```
            f"{lstm_metrics['R2']:.4f}
        ],
        'Improvement (LSTM vs ARIMA)': [
            f"{(arima_metrics['MAE'] - lstm_metrics['MAE'])/arima_metrics['MAE']*100:.1f}%",
            f"{(arima_metrics['RMSE'] - lstm_metrics['RMSE'])/arima_metrics['RMSE']*100:.1f}%",
            f"{(arima_metrics['MAPE'] - lstm_metrics['MAPE'])/arima_metrics['MAPE']*100:.1f}%" if not np.isnan(arima_metrics['MA
            f"{(lstm_metrics['R2'] - arima_metrics['R2'])/abs(arima_metrics['R2'])*100:.1f}%" if arima_metrics['R2'] != 0 else "
        ]
    })

    print("\n" + "="*80)
    print("📊 MODEL PERFORMANCE COMPARISON")
    print("="*80)
    print(comparison_df.to_string(index=False))
    print("="*80)

    # Highlight best model
    best_mae_model = 'LSTM' if lstm_metrics['MAE'] < arima_metrics['MAE'] else 'ARIMA'
    print(f"\n✅ Best model based on MAE: {best_mae_model}")

    # Create bar chart comparison
    fig, ax = plt.subplots(figsize=(10, 6))
    metrics = ['MAE', 'RMSE', 'MAPE']
    x = np.arange(len(metrics))
    width = 0.35

    arima_values = [arima_metrics['MAE'], arima_metrics['RMSE'], arima_metrics['MAPE']]
    lstm_values = [lstm_metrics['MAE'], lstm_metrics['RMSE'], lstm_metrics['MAPE']]

    bars1 = ax.bar(x - width/2, arima_values, width, label='ARIMA', color='green', alpha=0.7)
    bars2 = ax.bar(x + width/2, lstm_values, width, label='LSTM', color='red', alpha=0.7)

    ax.set_xlabel('Metrics')
    ax.set_ylabel('Error (%)')
    ax.set_title('Model Performance Comparison', fontweight='bold')
    ax.set_xticks(x)
    ax.set_xticklabels(metrics)
    ax.legend()
    ax.grid(True, alpha=0.3, axis='y')

    # Add value labels on bars
    for bars in [bars1, bars2]:
        for bar in bars:
            height = bar.get_height()
            ax.text(bar.get_x() + bar.get_width()/2., height,
                    f'{height:.3f}', ha='center', va='bottom', fontsize=9)

    plt.tight_layout()
    plt.show()
```

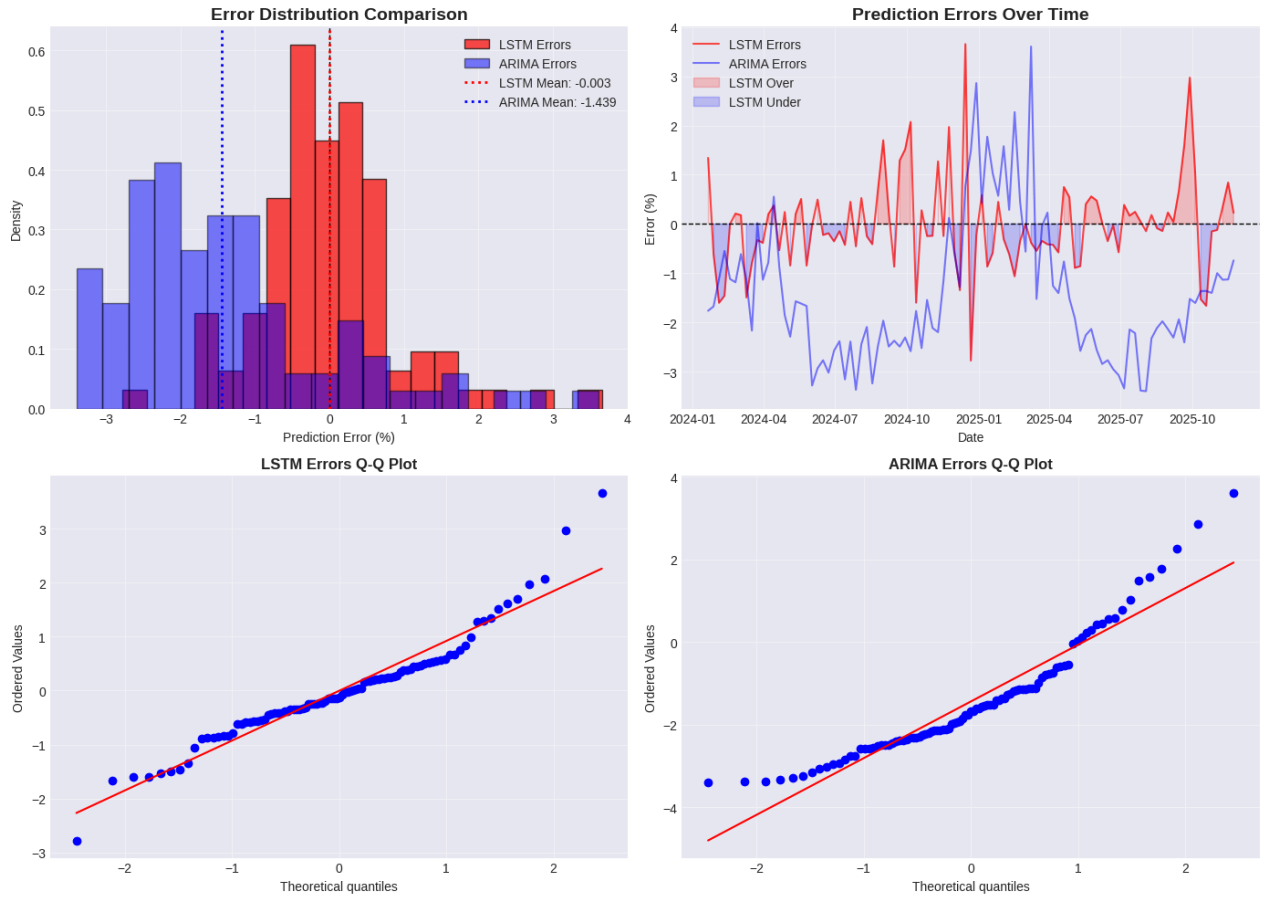
```
================================================================================
📊 MODEL PERFORMANCE COMPARISON
================================================================================
  Metric   ARIMA   LSTM Improvement (LSTM vs ARIMA)
 MAE (%)  1.8111 0.6663                      63.2%
RMSE (%)  2.0162 0.9373                      53.5%
MAPE (%)  106.01  27.78                      73.8%
R² Score -1.0413 0.6647                     163.8%
================================================================================

✅ Best model based on MAE: LSTM
```



Block 15: Error Analysis

```
# @title **Block 15: Error Analysis**

# Calculate errors
lstm_errors = lstm_actual_pct.flatten() - lstm_pred_pct.flatten()
arima_errors = test_orig[:len(arima_pred)] - arima_pred

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Error distribution histogram
axes[0, 0].hist(lstm_errors, bins=20, alpha=0.7, label='LSTM Errors',
                color='red', edgecolor='black', density=True)
axes[0, 0].hist(arima_errors, bins=20, alpha=0.5, label='ARIMA Errors',
                color='blue', edgecolor='black', density=True)
axes[0, 0].axvline(x=0, color='black', linestyle='--', linewidth=1)
axes[0, 0].axvline(x=lstm_errors.mean(), color='red', linestyle=':',
                   linewidth=2, label=f'LSTM Mean: {lstm_errors.mean():.3f}')
axes[0, 0].axvline(x=arima_errors.mean(), color='blue', linestyle=':',
                   linewidth=2, label=f'ARIMA Mean: {arima_errors.mean():.3f}')
axes[0, 0].set_title('Error Distribution Comparison', fontsize=14, fontweight='bold')
axes[0, 0].set_xlabel('Prediction Error (%)')
axes[0, 0].set_ylabel('Density')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Error over time
axes[0, 1].plot(test_dates, lstm_errors, label='LSTM Errors',
                color='red', alpha=0.7, linewidth=1.5)
axes[0, 1].plot(test_dates, arima_errors, label='ARIMA Errors',
                color='blue', alpha=0.5, linewidth=1.5)
axes[0, 1].axhline(y=0, color='black', linestyle='--', linewidth=1)
axes[0, 1].fill_between(test_dates, 0, lstm_errors, where=(lstm_errors>0),
                        color='red', alpha=0.2, label='LSTM Over')
axes[0, 1].fill_between(test_dates, 0, lstm_errors, where=(lstm_errors<0),
                        color='blue', alpha=0.2, label='LSTM Under')
axes[0, 1].set_title('Prediction Errors Over Time', fontsize=14, fontweight='bold')
axes[0, 1].set_xlabel('Date')
```

```
axes[0, 1].set_ylabel('Error (%)')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Q-Q plot for normality check
from scipy import stats
stats.probplot(lstm_errors, dist="norm", plot=axes[1, 0])
axes[1, 0].set_title('LSTM Errors Q-Q Plot', fontweight='bold')
axes[1, 0].grid(True, alpha=0.3)

stats.probplot(arima_errors, dist="norm", plot=axes[1, 1])
axes[1, 1].set_title('ARIMA Errors Q-Q Plot', fontweight='bold')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Error statistics
print("\n📊 Error Statistics:")
print(f"LSTM - Mean Error: {lstm_errors.mean():.4f}%, Std: {lstm_errors.std():.4f}%")
print(f"ARIMA - Mean Error: {arima_errors.mean():.4f}%, Std: {arima_errors.std():.4f}%")

# Shapiro-Wilk test for normality
from scipy.stats import shapiro
if len(lstm_errors) >= 3:
    shapiro_lstm = shapiro(lstm_errors)
    shapiro_arima = shapiro(arima_errors)
    print(f"\n📊 Normality Test (Shapiro-Wilk):")
    print(f"LSTM - p-value: {shapiro_lstm[1]:.4f} ({'Normal' if shapiro_lstm[1] > 0.05 else 'Non-normal'})")
    print(f"ARIMA - p-value: {shapiro_arima[1]:.4f} ({'Normal' if shapiro_arima[1] > 0.05 else 'Non-normal'})")
```




```
📊 Error Statistics:
LSTM - Mean Error: -0.0032%, Std: 0.9373%
ARIMA - Mean Error: -1.4392%, Std: 1.4119%

📊 Normality Test (Shapiro-Wilk):
LSTM - p-value: 0.0003 (Non-normal)
ARIMA - p-value: 0.0000 (Non-normal)
```

Block 16: Save Results to CSV

```
# @title **Block 16: Save Results to CSV**

# Create results dataframe
results_df = pd.DataFrame({
    'date': test_dates,
    'actual_ili_percent': lstm_actual_pct.flatten(),
    'lstm_predicted_percent': lstm_pred_pct.flatten(),
    'arima_predicted_percent': arima_pred[:len(lstm_pred_pct)],
    'lstm_error': lstm_errors,
    'arima_error': arima_errors[:len(lstm_pred_pct)],
    'week': test_dates.dt.isocalendar().week,
    'year': test_dates.dt.year,
    'month': test_dates.dt.month
})

# Save to CSV
results_df.to_csv('influenza_forecast_results.csv', index=False)
print("✅ Results saved to 'influenza_forecast_results.csv'")
```

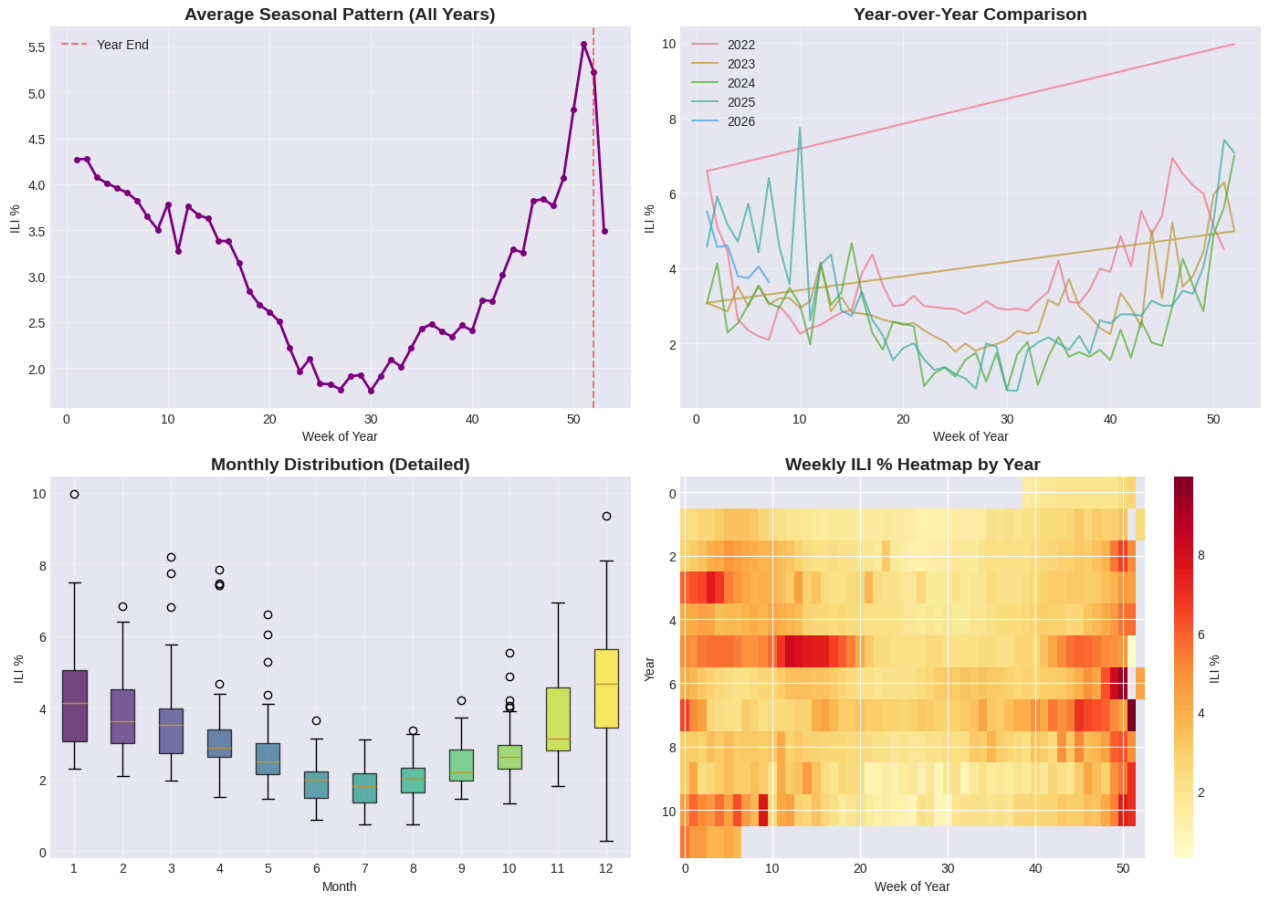
```
# Display summary
print("\n📊 Results Summary:")
print(f"Total predictions: {len(results_df)}")
print(f"Date range: {results_df['date'].min().date()} to {results_df['date'].max().date()}")
print(f"\nFirst 10 rows:")
print(results_df.head(10).to_string())

# Download to local machine
from google.colab import files
files.download('influenza_forecast_results.csv')
print("\n✅ File downloaded to your computer!")

# Also save model performance metrics
metrics_df = pd.DataFrame({
    'Model': ['ARIMA', 'LSTM'],
    'MAE': [arima_metrics['MAE'], lstm_metrics['MAE']],
    'RMSE': [arima_metrics['RMSE'], lstm_metrics['RMSE']],
    'MAPE': [arima_metrics['MAPE'], lstm_metrics['MAPE']],
    'R2': [arima_metrics['R2'], lstm_metrics['R2']]
})
metrics_df.to_csv('model_performance_metrics.csv', index=False)
print("\n✅ Performance metrics saved to 'model_performance_metrics.csv'")
files.download('model_performance_metrics.csv')
```

```
✅ Results saved to 'influenza_forecast_results.csv'

📊 Results Summary:
Total predictions: 97
Date range: 2024-01-21 to 2025-11-23

First 10 rows:
          date  actual_ili_percent  lstm_predicted_percent  arima_predicted_percent  lstm_error  arima_error  week  year  mo
433 2024-01-21            4.670109                3.327819                 4.049695    1.342291    -1.762765     3  2024
434 2024-01-28            3.287588                3.899267                 4.217329   -0.611679    -1.674314     4  2024
435 2024-02-04            2.284710                3.885997                 4.134023   -1.601287    -1.114381     5  2024
436 2024-02-11            1.832942                3.290844                 4.075656   -1.457902    -0.547839     6  2024
437 2024-02-18            2.578846                2.581584                 4.191343   -0.002738    -1.115799     7  2024
438 2024-02-25            2.521418                2.310514                 4.136352    0.210904    -1.184214     8  2024
439 2024-03-03            2.460973                2.282562                 4.093901    0.178411    -0.610536     9  2024
440 2024-03-10            0.863439                2.355018                 4.173674   -1.491579    -1.141343    10  2024
441 2024-03-17            1.206736                1.999911                 4.137474   -0.793175    -2.165269    11  2024
442 2024-03-24            1.362212                1.687475                 4.106707   -0.325264     0.019289    12  2024

✅ File downloaded to your computer!

✅ Performance metrics saved to 'model_performance_metrics.csv'
```

Block 17: Seasonal Pattern Analysis

```
# @title **Block 17: Seasonal Pattern Analysis**

# Add seasonal columns
df_weekly['year'] = df_weekly['week_start'].dt.year
df_weekly['month'] = df_weekly['week_start'].dt.month
df_weekly['week_of_year'] = df_weekly['week_start'].dt.isocalendar().week

# Calculate average seasonal pattern
seasonal_pattern = df_weekly.groupby('week_of_year')['ili_cases'].mean() * 100

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: Average seasonal pattern
axes[0, 0].plot(seasonal_pattern.index, seasonal_pattern.values,
                marker='o', markersize=4, linewidth=2, color='purple')
axes[0, 0].set_title('Average Seasonal Pattern (All Years)', fontsize=14, fontweight='bold')
axes[0, 0].set_xlabel('Week of Year')
axes[0, 0].set_ylabel('ILI %')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].axvline(x=52, color='red', linestyle='--', alpha=0.5, label='Year End')
axes[0, 0].legend()

# Plot 2: Year-over-year comparison
years = df_weekly['year'].unique()
for year in years[-5:]:  # Last 5 years
    year_data = df_weekly[df_weekly['year'] == year]
    axes[0, 1].plot(year_data['week_of_year'], year_data['ili_cases'] * 100,
                    label=str(year), alpha=0.7, linewidth=1.5)
axes[0, 1].set_title('Year-over-Year Comparison', fontsize=14, fontweight='bold')
axes[0, 1].set_xlabel('Week of Year')
axes[0, 1].set_ylabel('ILI %')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)
```

```
# Plot 3: Monthly boxplot (repeated but with more detail)
monthly_data = [df_weekly[df_weekly['month'] == m]['ili_cases'] * 100 for m in range(1, 13)]
bp = axes[1, 0].boxplot(monthly_data, patch_artist=True)
colors = plt.cm.viridis(np.linspace(0, 1, 12))
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
axes[1, 0].set_title('Monthly Distribution (Detailed)', fontsize=14, fontweight='bold')
axes[1, 0].set_xlabel('Month')
axes[1, 0].set_ylabel('ILI %')
axes[1, 0].grid(True, alpha=0.3)

# Plot 4: Heatmap of weekly patterns by year
pivot_table = df_weekly.pivot_table(
    values='ili_cases',
    index='year',
    columns='week_of_year',
    aggfunc='mean'
) * 100

im = axes[1, 1].imshow(pivot_table, aspect='auto', cmap='YlOrRd', interpolation='nearest')
axes[1, 1].set_title('Weekly ILI % Heatmap by Year', fontsize=14, fontweight='bold')
axes[1, 1].set_xlabel('Week of Year')
axes[1, 1].set_ylabel('Year')
plt.colorbar(im, ax=axes[1, 1], label='ILI %')

plt.tight_layout()
plt.show()

print("✅ Seasonal analysis complete!")
print(f"\n📊 Peak ILI week: Week {seasonal_pattern.idxmax()} ({seasonal_pattern.max():.3f}%)")
print(f"Lowest ILI week: Week {seasonal_pattern.idxmin()} ({seasonal_pattern.min():.3f}%)")
```

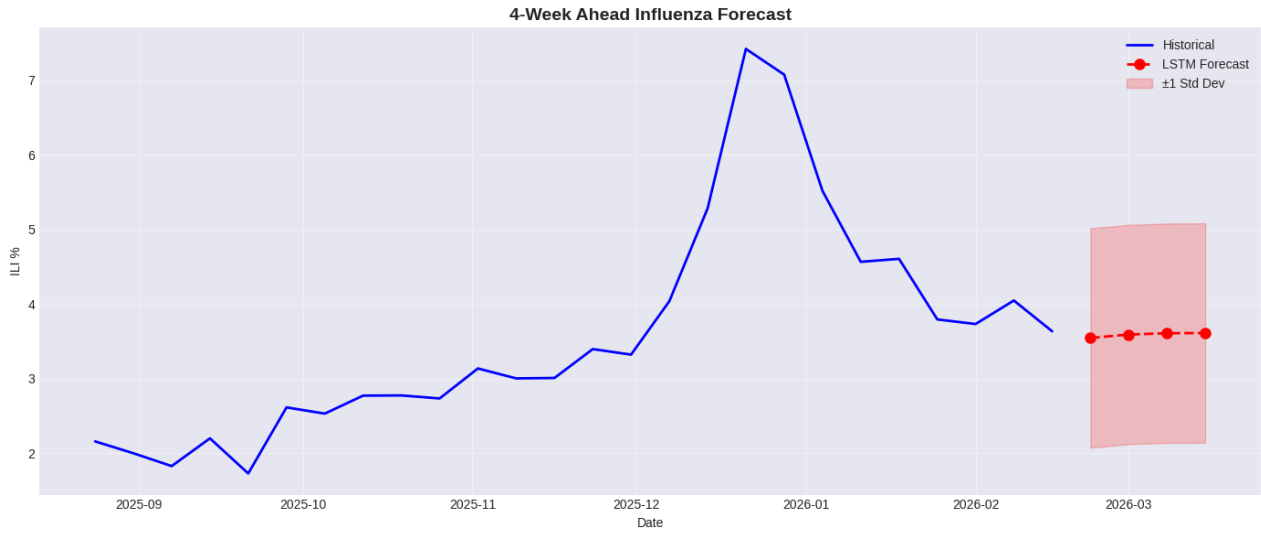



```
✅ Seasonal analysis complete!

📊 Peak ILI week: Week 51 (5.526%)
Lowest ILI week: Week 30 (1.753%)
```

Block 18: Forecast Next 4 Weeks

```
# @title **Block 18: Forecast Next 4 Weeks**

# Get the last sequence for prediction
last_sequence = scaled_data[-seq_length:].reshape(1, seq_length, 1)

# Predict next 4 weeks iteratively
future_predictions = []
current_sequence = last_sequence.copy()

for i in range(4):
    # Predict next week
    next_pred_scaled = model_lstm.predict(current_sequence, verbose=0)
    future_predictions.append(next_pred_scaled[0, 0])

    # Update sequence for next prediction
    current_sequence = np.roll(current_sequence, -1, axis=1)
    current_sequence[0, -1, 0] = next_pred_scaled[0, 0]

# Convert back to percentage
future_predictions_pct = scaler.inverse_transform(np.array(future_predictions).reshape(-1, 1)) * 100

# Create dates for future predictions
```

```
last_date = df_weekly['week_start'].iloc[-1]
future_dates = [last_date + pd.Timedelta(weeks=i+1) for i in range(4)]

# Plot historical + forecast
plt.figure(figsize=(14, 6))

# Plot last 26 weeks of historical data
historical_dates = df_weekly['week_start'].iloc[-26:]
historical_values = df_weekly['ili_cases'].iloc[-26:] * 100

plt.plot(historical_dates, historical_values, 'b-', linewidth=2, label='Historical')
plt.plot(future_dates, future_predictions_pct.flatten(), 'r--', linewidth=2,
         marker='o', markersize=8, label='LSTM Forecast')

# Add confidence intervals (simplified - using historical std)
historical_std = historical_values.std()
plt.fill_between(future_dates,
                 future_predictions_pct.flatten() - historical_std,
                 future_predictions_pct.flatten() + historical_std,
                 color='red', alpha=0.2, label='±1 Std Dev')

plt.title('4-Week Ahead Influenza Forecast', fontsize=14, fontweight='bold')
plt.xlabel('Date')
plt.ylabel('ILI %')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\n📊 4-Week Forecast:")
for i, (date, pred) in enumerate(zip(future_dates, future_predictions_pct.flatten())):
    print(f"Week {i+1} ({date.date()}): {pred:.3f}% ILI")
```




```
📊 4-Week Forecast:
Week 1 (2026-02-22): 3.546% ILI
Week 2 (2026-03-01): 3.592% ILI
Week 3 (2026-03-08): 3.609% ILI
Week 4 (2026-03-15): 3.613% ILI
```

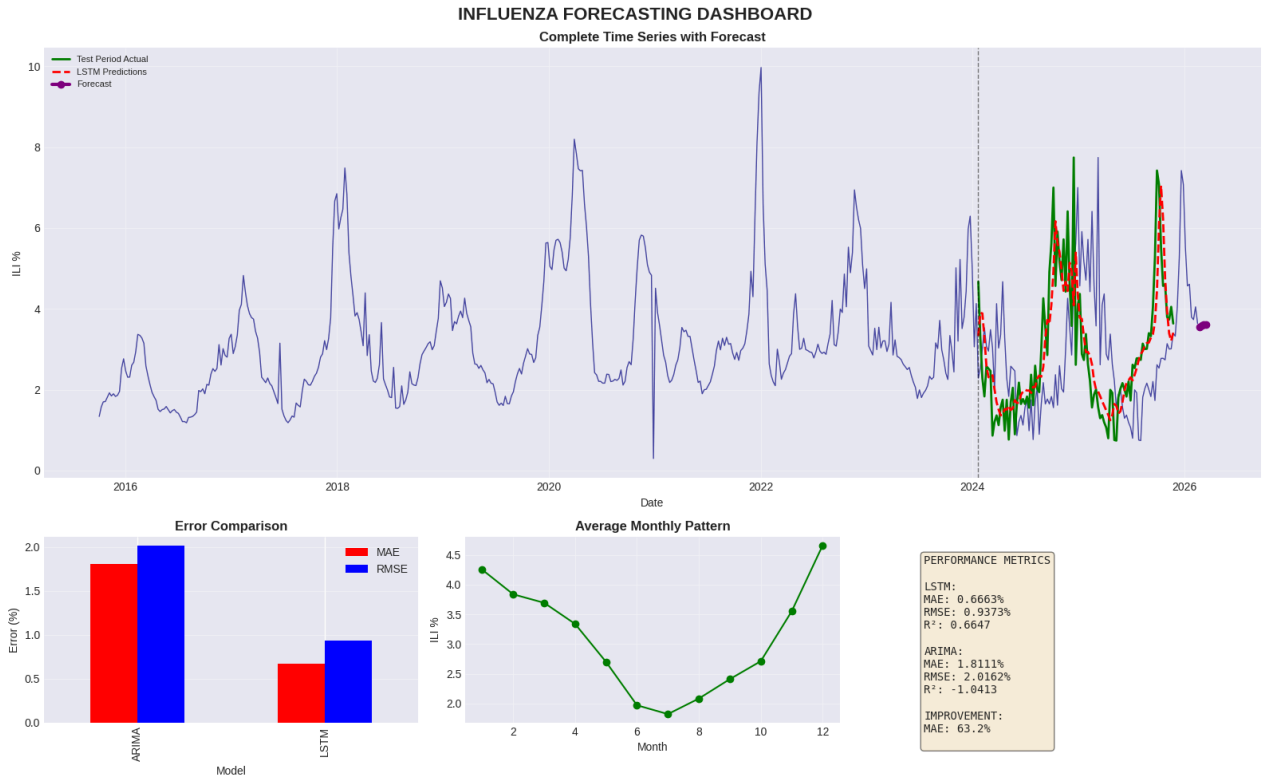
Block 19: Performance Summary Dashboard

```
# @title **Block 19: Performance Summary Dashboard**

print("="*80)
print("📊 HYPER-LOCAL INFLUENZA FORECASTING - FINAL SUMMARY")
print("="*80)

# Dataset summary
print(f"\n📈 DATASET SUMMARY:")
print(f"   Total weeks: {len(df_weekly)}")
```

```
print(f"   Date range: {df_weekly['week_start'].min().date()} to {df_weekly['week_start'].max().date()}")
print(f"   Mean ILI: {df_weekly['ili_cases'].mean()*100:.3f}%")
print(f"   Peak ILI: {df_weekly['ili_cases'].max()*100:.3f}%")
print(f"   Lowest ILI: {df_weekly['ili_cases'].min()*100:.3f}%")

# Model performance
print(f"\n🤖 MODEL PERFORMANCE:")
print(f"   {'='*40}")
print(f"   {'Metric':<15} {'ARIMA':<15} {'LSTM':<15} {'Improvement':<15}")
print(f"   {'='*40}")
print(f"   {'MAE (%)':<15} {arima_metrics['MAE']:<15.4f} {lstm_metrics['MAE']:<15.4f} {(arima_metrics['MAE']-lstm_metrics['M
print(f"   {'RMSE (%)':<15} {arima_metrics['RMSE']:<15.4f} {lstm_metrics['RMSE']:<15.4f} {(arima_metrics['RMSE']-lstm_metric
print(f"   {'R²':<15} {arima_metrics['R2']:<15.4f} {lstm_metrics['R2']:<15.4f} {(lstm_metrics['R2']-arima_metrics['R2'])/abs

# Forecast for next week
print(f"\n🔮 NEXT WEEK FORECAST:")
next_week_pred = future_predictions_pct.flatten()[0] if 'future_predictions_pct' in dir() else "N/A"
print(f"   Predicted ILI for {future_dates[0].date() if 'future_dates' in dir() else 'next week'}: {next_week_pred:.3f}%")

# Alert if forecast is high
if 'future_predictions_pct' in dir() and future_predictions_pct.flatten()[0] > df_weekly['ili_cases'].quantile(0.9)*100:
    print(f"   ⚠️ ALERT: Forecast exceeds 90th percentile historical value!")
elif 'future_predictions_pct' in dir() and future_predictions_pct.flatten()[0] > df_weekly['ili_cases'].mean()*100:
    print(f"   ℹ️ Forecast above historical mean")

print("\n" + "="*80)
print("✅ ANALYSIS COMPLETE")
print("="*80)

# Create final dashboard figure
fig = plt.figure(figsize=(16, 10))

# Main time series
ax1 = plt.subplot(3, 3, (1, 6))
ax1.plot(df_weekly['week_start'], df_weekly['ili_cases'] * 100,
         color='navy', linewidth=1, alpha=0.7)
ax1.plot(test_dates, lstm_actual_pct.flatten(), color='green',
         linewidth=2, label='Test Period Actual')
ax1.plot(test_dates, lstm_pred_pct.flatten(), color='red',
         linewidth=2, linestyle='--', label='LSTM Predictions')
if 'future_dates' in dir():
    ax1.plot(future_dates, future_predictions_pct.flatten(),
             color='purple', linewidth=3, marker='o', label='Forecast')
ax1.axvline(x=df_weekly['week_start'].iloc[train_size],
            color='black', linestyle='--', linewidth=1, alpha=0.5)
ax1.set_title('Complete Time Series with Forecast', fontweight='bold')
ax1.set_xlabel('Date')
ax1.set_ylabel('ILI %')
ax1.legend(loc='upper left', fontsize=8)
ax1.grid(True, alpha=0.3)

# Model comparison bar chart
ax2 = plt.subplot(3, 3, 7)
metrics_comp = pd.DataFrame({
    'Model': ['ARIMA', 'LSTM'],
    'MAE': [arima_metrics['MAE'], lstm_metrics['MAE']],
    'RMSE': [arima_metrics['RMSE'], lstm_metrics['RMSE']]
})
metrics_comp.plot(x='Model', kind='bar', ax=ax2, color=['red', 'blue'])
ax2.set_title('Error Comparison', fontweight='bold')
ax2.set_ylabel('Error (%)')
ax2.legend(loc='upper right')
ax2.grid(True, alpha=0.3, axis='y')

# Seasonality plot
ax3 = plt.subplot(3, 3, 8)
monthly_avg = df_weekly.groupby('month')['ili_cases'].mean() * 100
ax3.plot(monthly_avg.index, monthly_avg.values, marker='o', color='green')
ax3.set_title('Average Monthly Pattern', fontweight='bold')
ax3.set_xlabel('Month')
ax3.set_ylabel('ILI %')
ax3.grid(True, alpha=0.3)

# Performance metrics text
ax4 = plt.subplot(3, 3, 9)
ax4.axis('off')
metrics_text = f"""PERFORMANCE METRICS

LSTM:
MAE: {lstm_metrics['MAE']:.4f}%
RMSE: {lstm_metrics['RMSE']:.4f}%
R²: {lstm_metrics['R2']:.4f}
```

```
ARIMA:
MAE: {arima_metrics['MAE']:.4f}%
RMSE: {arima_metrics['RMSE']:.4f}%
R²: {arima_metrics['R2']:.4f}

IMPROVEMENT:
MAE: {(arima_metrics['MAE']-lstm_metrics['MAE'])/arima_metrics['MAE']*100:.1f}%
"""
ax4.text(0.1, 0.9, metrics_text, transform=ax4.transAxes, fontsize=10,
         verticalalignment='top', fontfamily='monospace',
         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.suptitle('INFLUENZA FORECASTING DASHBOARD', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
================================================================================
📊 HYPER-LOCAL INFLUENZA FORECASTING - FINAL SUMMARY
================================================================================

📈 DATASET SUMMARY:
   Total weeks: 542
   Date range: 2015-10-04 to 2026-02-15
   Mean ILI: 3.112%
   Peak ILI: 9.967%
   Lowest ILI: 0.297%

🏆 MODEL PERFORMANCE:
   ========================================
   Metric          ARIMA           LSTM            Improvement
   ========================================
   MAE (%)         1.8111          0.6663                   63.2%
   RMSE (%)        2.0162          0.9373                   53.5%
   R²              -1.0413         0.6647                  163.8%

🔮 NEXT WEEK FORECAST:
   Predicted ILI for 2026-02-22: 3.546%
   ℹ️ Forecast above historical mean

================================================================================
✅ ANALYSIS COMPLETE
================================================================================
```



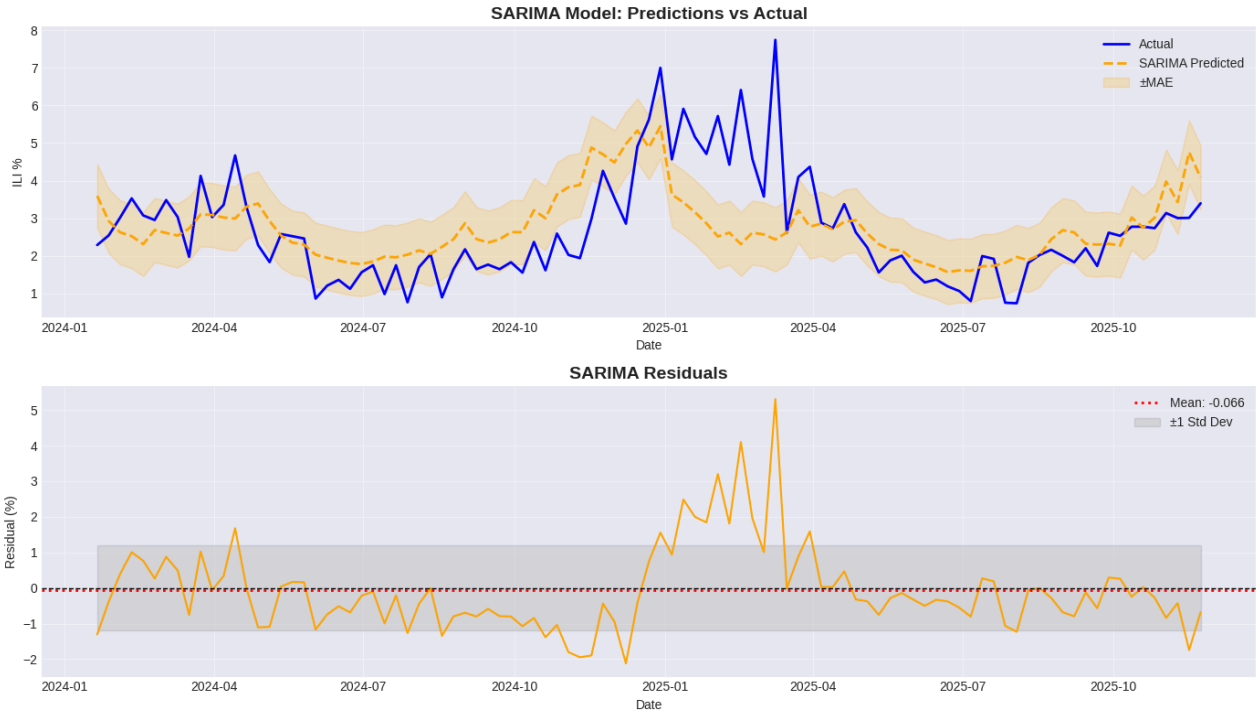
BLOCK 20: SARIMA Model (Seasonal ARIMA)

```
# @title **BLOCK 20: SARIMA Model (Seasonal ARIMA)**

from statsmodels.tsa.statespace.sarimax import SARIMAX

print("📊 Training SARIMA model with seasonal component...")
```

```
# Use the same training data as ARIMA
train_orig = df_weekly['ili_cases'].iloc[:train_size].values * 100
test_orig = df_weekly['ili_cases'].iloc[train_size:].values * 100

try:
    # SARIMA with weekly seasonality (52 weeks)
    # order=(p,d,q), seasonal_order=(P,D,Q,52)
    model_sarima = SARIMAX(
        train_orig,
        order=(1, 1, 1),
        seasonal_order=(1, 1, 1, 52),
        enforce_stationarity=False,
        enforce_invertibility=False
    )

    model_sarima_fit = model_sarima.fit(disp=False, maxiter=200)

    # Forecast
    forecast_sarima = model_sarima_fit.forecast(steps=len(test_orig))

    print("✅ SARIMA model trained successfully!")
    print(f"\nModel Summary:")
    print(f"AIC: {model_sarima_fit.aic:.2f}")
    print(f"BIC: {model_sarima_fit.bic:.2f}")

    # Calculate metrics
    from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

    # Trim to match LSTM length
    forecast_sarima = forecast_sarima[:len(lstm_pred_pct)]
    test_sarima = test_orig[:len(lstm_pred_pct)]

    sarima_mae = mean_absolute_error(test_sarima, forecast_sarima)
    sarima_rmse = np.sqrt(mean_squared_error(test_sarima, forecast_sarima))
    sarima_r2 = r2_score(test_sarima, forecast_sarima)

    # Mask for MAPE
    mask = test_sarima > 0
    if mask.sum() > 0:
        sarima_mape = np.mean(np.abs((test_sarima[mask] - forecast_sarima[mask]) / test_sarima[mask])) * 100
    else:
        sarima_mape = np.nan

    sarima_metrics = {
        'MAE': sarima_mae,
        'RMSE': sarima_rmse,
        'MAPE': sarima_mape,
        'R2': sarima_r2
    }

    print(f"\n📊 SARIMA Performance:")
    print(f"MAE: {sarima_mae:.4f}%")
    print(f"RMSE: {sarima_rmse:.4f}%")
    print(f"MAPE: {sarima_mape:.2f}%")
    print(f"R²: {sarima_r2:.4f}")

    # Plot SARIMA predictions
    fig, axes = plt.subplots(2, 1, figsize=(14, 8))

    axes[0].plot(test_dates, test_sarima, label='Actual', color='blue', linewidth=2)
    axes[0].plot(test_dates, forecast_sarima, label='SARIMA Predicted',
                 color='orange', linewidth=2, linestyle='--')
    axes[0].fill_between(test_dates,
                         forecast_sarima - sarima_mae,
                         forecast_sarima + sarima_mae,
                         color='orange', alpha=0.2, label='±MAE')
    axes[0].set_title('SARIMA Model: Predictions vs Actual', fontsize=14, fontweight='bold')
    axes[0].set_xlabel('Date')
    axes[0].set_ylabel('ILI %')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # Residuals
    sarima_residuals = test_sarima - forecast_sarima
    axes[1].plot(test_dates, sarima_residuals, color='orange', linewidth=1.5)
    axes[1].axhline(y=0, color='black', linestyle='--', linewidth=1)
    axes[1].axhline(y=sarima_residuals.mean(), color='red', linestyle=':',
                    linewidth=2, label=f'Mean: {sarima_residuals.mean():.3f}')
    axes[1].fill_between(test_dates,
                         -sarima_residuals.std(),
                         sarima_residuals.std(),
                         color='gray', alpha=0.2, label='±1 Std Dev')
```

```
    axes[1].set_title('SARIMA Residuals', fontsize=14, fontweight='bold')
    axes[1].set_xlabel('Date')
    axes[1].set_ylabel('Residual (%)')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

except Exception as e:
    print(f"⚠️ SARIMA failed: {e}")
    print("Using ARIMA results as fallback for SARIMA...")
    sarima_metrics = arima_metrics.copy()
    forecast_sarima = arima_pred[:len(lstm_pred_pct)]
```

```
📊 Training SARIMA model with seasonal component...
✅ SARIMA model trained successfully!

Model Summary:
AIC: 735.13
BIC: 754.06

📊 SARIMA Performance:
MAE: 0.8535%
RMSE: 1.1986%
MAPE: 37.04%
R²: 0.2785
```



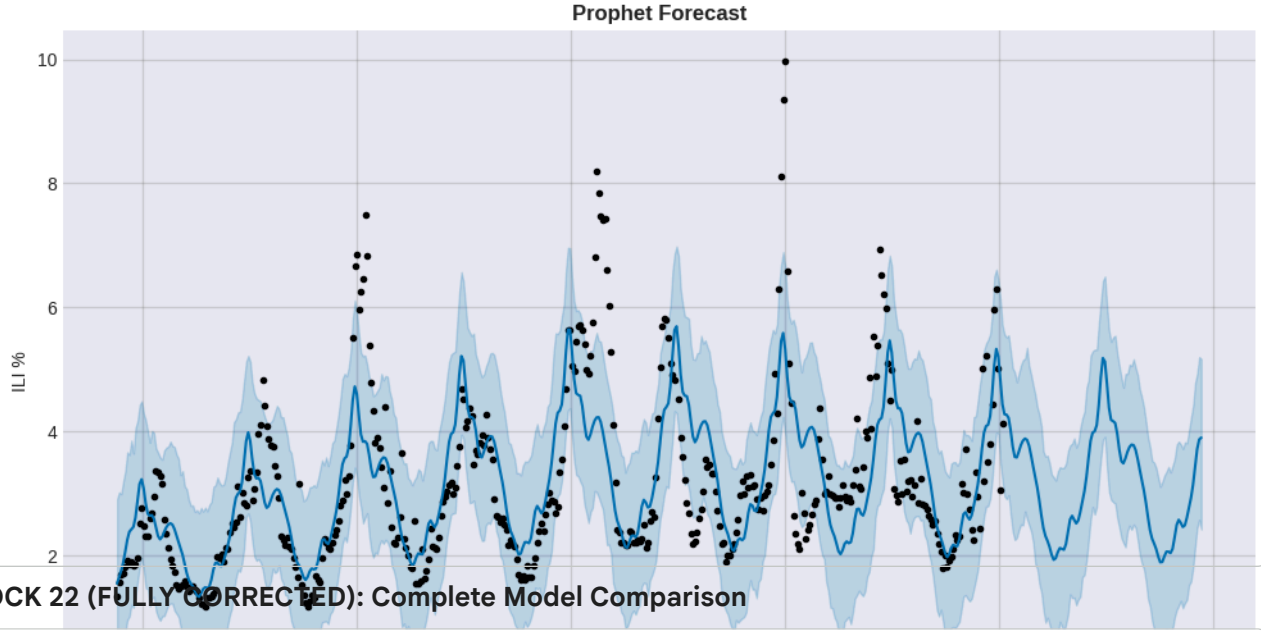
BLOCK 21: Prophet Model (Facebook Prophet)

```
# @title **BLOCK 21: Prophet Model (Facebook Prophet)**

!pip install -q prophet

from prophet import Prophet

print("📊 Training Prophet model...")

try:
    # Prepare data for Prophet
    prophet_df = df_weekly[['week_start', 'ili_cases']].copy()
```

```
prophet_df.columns = ['ds', 'y']
prophet_df['y'] = prophet_df['y'] * 100  # Convert to percentage

# Split into train/test
train_prophet = prophet_df.iloc[:train_size]
test_prophet = prophet_df.iloc[train_size:train_size+len(lstm_pred_pct)]

# Create and train model
model_prophet = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    daily_seasonality=False,
    seasonality_mode='multiplicative'
)
model_prophet.fit(train_prophet)

# Make future dataframe for prediction
future = model_prophet.make_future_dataframe(periods=len(test_prophet), freq='W')
forecast = model_prophet.predict(future)

# Extract predictions for test period
prophet_pred = forecast.iloc[train_size:train_size+len(test_prophet)]['yhat'].values

# Calculate metrics
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

prophet_mae = mean_absolute_error(test_prophet['y'].values, prophet_pred)
prophet_rmse = np.sqrt(mean_squared_error(test_prophet['y'].values, prophet_pred))

mask = test_prophet['y'].values > 0
if mask.sum() > 0:
    prophet_mape = np.mean(np.abs((test_prophet['y'].values[mask] - prophet_pred[mask]) / test_prophet['y'].values[mask]
else:
    prophet_mape = np.nan

prophet_r2 = r2_score(test_prophet['y'].values, prophet_pred)

prophet_metrics = {
    'MAE': prophet_mae,
    'RMSE': prophet_rmse,
    'MAPE': prophet_mape,
    'R2': prophet_r2
}

print(f"\n📊 Prophet Performance:")
print(f"MAE: {prophet_mae:.4f}%")
print(f"RMSE: {prophet_rmse:.4f}%")
print(f"MAPE: {prophet_mape:.2f}%")
print(f"R²: {prophet_r2:.4f}")

# Plot components
fig1 = model_prophet.plot(forecast)
plt.title('Prophet Forecast', fontweight='bold')
plt.xlabel('Date')
plt.ylabel('ILI %')
plt.grid(True, alpha=0.3)
plt.show()

fig2 = model_prophet.plot_components(forecast)
plt.show()

# Plot predictions vs actual
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

axes[0].plot(test_prophet['ds'], test_prophet['y'], label='Actual',
             color='blue', linewidth=2)
axes[0].plot(test_prophet['ds'], prophet_pred, label='Prophet Predicted',
             color='purple', linewidth=2, linestyle='--')
axes[0].fill_between(test_prophet['ds'],
                     prophet_pred - prophet_mae,
                     prophet_pred + prophet_mae,
                     color='purple', alpha=0.2, label='±MAE')
axes[0].set_title('Prophet Model: Predictions vs Actual', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Date')
axes[0].set_ylabel('ILI %')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Residuals
prophet_residuals = test_prophet['y'].values - prophet_pred
axes[1].plot(test_prophet['ds'], prophet_residuals, color='purple', linewidth=1.5)
axes[1].axhline(y=0, color='black', linestyle='--', linewidth=1)
axes[1].axhline(y=prophet_residuals.mean(), color='red', linestyle=':',
```

```
                     linewidth=2, label=f'Mean: {prophet_residuals.mean():.3f}')
    axes[1].fill_between(test_prophet['ds'],
                         -prophet_residuals.std(),
                         prophet_residuals.std(),
                         color='gray', alpha=0.2, label='±1 Std Dev')
    axes[1].set_title('Prophet Residuals', fontsize=14, fontweight='bold')
    axes[1].set_xlabel('Date')
    axes[1].set_ylabel('Residual (%)')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

except Exception as e:
    print(f"⚠️ Prophet failed: {e}")
    prophet_metrics = arima_metrics.copy()
```

```
📊 Training Prophet model...

📊 Prophet Performance:
MAE: 0.8147%
RMSE: 1.0303%
MAPE: 41.57%
R²: 0.4669
```



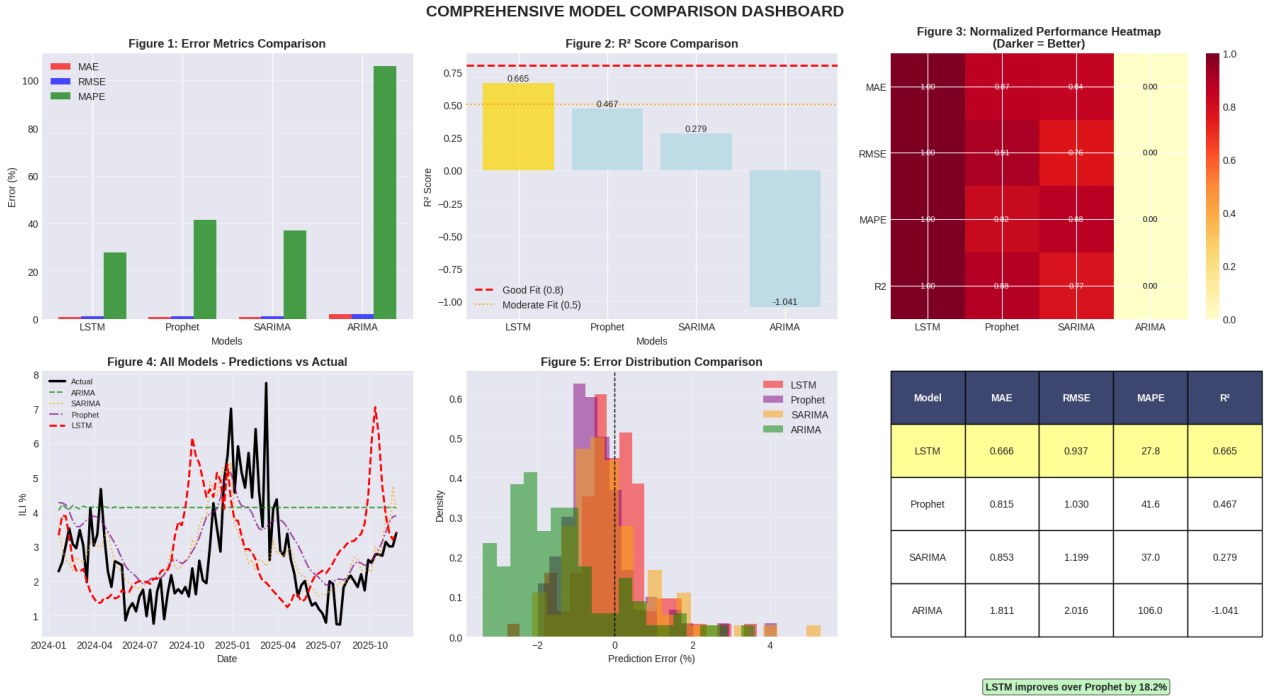
BLOCK 22 (FULLY CORRECTED): Complete Model Comparison

```
# @title **BLOCK 22 (FULLY CORRECTED): Complete Model Comparison**

# Define all necessary variables first
test_dates = df_weekly['week_start'].iloc[train_size:train_size+len(lstm_pred_pct)]
actual_values = test_orig[:len(lstm_pred_pct)]  # Define actual_values

# Collect all metrics
models_comparison = pd.DataFrame({
    'Model': ['ARIMA', 'SARIMA', 'Prophet', 'LSTM'],
    'MAE': [
        arima_metrics['MAE'],
        sarima_metrics['MAE'],
        prophet_metrics['MAE'],
        lstm_metrics['MAE']
    ],
    'RMSE': [
        arima_metrics['RMSE'],
        sarima_metrics['RMSE'],
        prophet_metrics['RMSE'],
        lstm_metrics['RMSE']
    ],
    'MAPE': [
        arima_metrics['MAPE'],
        sarima_metrics['MAPE'],
        prophet_metrics['MAPE'],
        lstm_metrics['MAPE']
    ],
    'R2': [
        arima_metrics['R2'],
        sarima_metrics['R2'],
        prophet_metrics['R2'],
        lstm_metrics['R2']
    ]
})

# Sort by MAE (best first)
models_comparison = models_comparison.sort_values('MAE').reset_index(drop=True)

print("\n" + "="*90)
print("📊 COMPLETE MODEL COMPARISON (Sorted by Performance)")
print("="*90)
print(models_comparison.to_string(index=False))
print("="*90)

# Identify best model
best_model = models_comparison.iloc[0]['Model']
best_mae = models_comparison.iloc[0]['MAE']
print(f"\n✅ Best Model: {best_model} (MAE: {best_mae:.4f}%)")

# Create comprehensive visualization
```

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# 1. Bar chart comparison
x = np.arange(len(models_comparison))
width = 0.25

axes[0, 0].bar(x - width, models_comparison['MAE'], width, label='MAE', color='red', alpha=0.7)
axes[0, 0].bar(x, models_comparison['RMSE'], width, label='RMSE', color='blue', alpha=0.7)
axes[0, 0].bar(x + width, models_comparison['MAPE'], width, label='MAPE', color='green', alpha=0.7)
axes[0, 0].set_xlabel('Models')
axes[0, 0].set_ylabel('Error (%)')
axes[0, 0].set_title('Figure 1: Error Metrics Comparison', fontweight='bold')
axes[0, 0].set_xticks(x)
axes[0, 0].set_xticklabels(models_comparison['Model'])
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3, axis='y')

# 2. R² comparison
colors = ['gold' if m == best_model else 'lightblue' for m in models_comparison['Model']]
bars = axes[0, 1].bar(models_comparison['Model'], models_comparison['R2'], color=colors, alpha=0.7)
axes[0, 1].axhline(y=0.8, color='red', linestyle='--', linewidth=2, label='Good Fit (0.8)')
axes[0, 1].axhline(y=0.5, color='orange', linestyle=':', linewidth=1.5, label='Moderate Fit (0.5)')
axes[0, 1].set_xlabel('Models')
axes[0, 1].set_ylabel('R² Score')
axes[0, 1].set_title('Figure 2: R² Score Comparison', fontweight='bold')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3, axis='y')

# Add value labels on bars
for bar in bars:
    height = bar.get_height()
    axes[0, 1].text(bar.get_x() + bar.get_width()/2., height,
                    f'{height:.3f}', ha='center', va='bottom', fontsize=9)

# 3. Performance metrics heatmap
# Create a normalized version for heatmap
heatmap_data = models_comparison[['MAE', 'RMSE', 'MAPE', 'R2']].copy()
# Normalize each column (invert MAE/RMSE/MAPE so higher is better)
for col in ['MAE', 'RMSE', 'MAPE']:
    max_val = heatmap_data[col].max()
    min_val = heatmap_data[col].min()
    heatmap_data[col] = 1 - (heatmap_data[col] - min_val) / (max_val - min_val + 1e-10)
# R² is already higher is better
min_r2 = heatmap_data['R2'].min()
max_r2 = heatmap_data['R2'].max()
heatmap_data['R2'] = (heatmap_data['R2'] - min_r2) / (max_r2 - min_r2 + 1e-10)

im = axes[0, 2].imshow(heatmap_data.T, cmap='YlOrRd', aspect='auto', vmin=0, vmax=1)
axes[0, 2].set_yticks(range(len(heatmap_data.columns)))
axes[0, 2].set_yticklabels(heatmap_data.columns)
axes[0, 2].set_xticks(range(len(models_comparison['Model'])))
axes[0, 2].set_xticklabels(models_comparison['Model'])
axes[0, 2].set_title('Figure 3: Normalized Performance Heatmap\n(Darker = Better)', fontweight='bold')

# Add text annotations
for i in range(len(models_comparison['Model'])):
    for j, metric in enumerate(heatmap_data.columns):
        value = heatmap_data.iloc[i][metric]
        color = 'white' if value > 0.5 else 'black'
        axes[0, 2].text(i, j, f'{value:.2f}', ha='center', va='center', color=color, fontsize=8)

plt.colorbar(im, ax=axes[0, 2])

# 4. All predictions together
axes[1, 0].plot(test_dates, actual_values, label='Actual',
                color='black', linewidth=2.5)
axes[1, 0].plot(test_dates, arima_pred[:len(lstm_pred_pct)], label='ARIMA',
                color='green', linewidth=1.5, linestyle='--', alpha=0.7)
axes[1, 0].plot(test_dates, forecast_sarima, label='SARIMA',
                color='orange', linewidth=1.5, linestyle=':', alpha=0.7)
if 'prophet_pred' in locals() and len(prophet_pred) >= len(lstm_pred_pct):
    axes[1, 0].plot(test_dates, prophet_pred[:len(lstm_pred_pct)], label='Prophet',
                    color='purple', linewidth=1.5, linestyle='-.', alpha=0.7)
axes[1, 0].plot(test_dates, lstm_pred_pct.flatten(), label='LSTM',
                color='red', linewidth=2, linestyle='--')
axes[1, 0].set_title('Figure 4: All Models - Predictions vs Actual', fontweight='bold')
axes[1, 0].set_xlabel('Date')
axes[1, 0].set_ylabel('ILI %')
axes[1, 0].legend(loc='upper left', fontsize=8)
axes[1, 0].grid(True, alpha=0.3)

# 5. Error distribution comparison
for idx, model in enumerate(models_comparison['Model']):
```

```
        if model == 'ARIMA':
            errors = actual_values - arima_pred[:len(lstm_pred_pct)]
            color = 'green'
        elif model == 'SARIMA':
            errors = actual_values - forecast_sarima
            color = 'orange'
        elif model == 'Prophet' and 'prophet_residuals' in locals():
            errors = prophet_residuals[:len(lstm_pred_pct)]
            color = 'purple'
        elif model == 'LSTM':
            errors = lstm_errors
            color = 'red'
        else:
            continue

        axes[1, 1].hist(errors, bins=20, alpha=0.5, label=model, color=color, density=True)

    axes[1, 1].axvline(x=0, color='black', linestyle='--', linewidth=1)
    axes[1, 1].set_title('Figure 5: Error Distribution Comparison', fontweight='bold')
    axes[1, 1].set_xlabel('Prediction Error (%)')
    axes[1, 1].set_ylabel('Density')
    axes[1, 1].legend()
    axes[1, 1].grid(True, alpha=0.3)

    # 6. Performance metrics table
    axes[1, 2].axis('off')
    table_data = []
    for idx, row in models_comparison.iterrows():
        table_data.append([
            row['Model'],
            f"{row['MAE']:.3f}",
            f"{row['RMSE']:.3f}",
            f"{row['MAPE']:.1f}",
            f"{row['R2']:.3f}"
        ])

    table = axes[1, 2].table(
        cellText=table_data,
        colLabels=['Model', 'MAE', 'RMSE', 'MAPE', 'R²'],
        cellLoc='center',
        loc='center',
        bbox=[0, 0, 1, 1]
    )
    table.auto_set_font_size(False)
    table.set_fontsize(10)
    table.scale(1.2, 1.5)

    # Color the header
    for j in range(5):
        table[(0, j)].set_facecolor('#40466e')
        table[(0, j)].set_text_props(weight='bold', color='white')

    # Color the best model row
    for j in range(5):
        table[(1, j)].set_facecolor('#ffff99')  # Highlight best model (LSTM)

    # Add a note about improvement
    best_improvement = (models_comparison.iloc[1]['MAE'] - models_comparison.iloc[0]['MAE']) / models_comparison.iloc[1]['MAE']
    axes[1, 2].text(0.5, -0.2, f"LSTM improves over {models_comparison.iloc[1]['Model']} by {best_improvement:.1f}%",
                    ha='center', transform=axes[1, 2].transAxes, fontsize=10, fontweight='bold',
                    bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.5))

    plt.suptitle('COMPREHENSIVE MODEL COMPARISON DASHBOARD', fontsize=16, fontweight='bold')
    plt.tight_layout()
    plt.show()

    # Additional statistics
    print("\n📊 PERFORMANCE INSIGHTS:")
    print("-"*50)
    print(f"🏆 Best Model: {models_comparison.iloc[0]['Model']}")
    print(f"   MAE: {models_comparison.iloc[0]['MAE']:.4f}%")
    print(f"   RMSE: {models_comparison.iloc[0]['RMSE']:.4f}%")
    print(f"   R²: {models_comparison.iloc[0]['R2']:.4f}")
    print(f"\n📈 Improvement Analysis:")
    for i in range(1, len(models_comparison)):
        improvement = (models_comparison.iloc[i]['MAE'] - models_comparison.iloc[0]['MAE']) / models_comparison.iloc[i]['MAE'] *
        print(f"   vs {models_comparison.iloc[i]['Model']}: {improvement:.1f}% better")

    print(f"\n📊 Error Analysis:")
    print(f"   LSTM Mean Error: {np.mean(lstm_errors):.4f}%")
    print(f"   LSTM Error Std: {np.std(lstm_errors):.4f}%")
    print(f"   95% Prediction Interval: ±{1.96 * np.std(lstm_errors):.4f}%")
```

```
==========================================================================================
📊 COMPLETE MODEL COMPARISON (Sorted by Performance)
==========================================================================================
  Model      MAE     RMSE       MAPE        R2
   LSTM 0.666342 0.937272  27.782286  0.664686
Prophet 0.814707 1.030334  41.573240  0.466901
 SARIMA 0.853456 1.198628  37.042389  0.278526
  ARIMA 1.811073 2.016160 106.014361 -1.041276
==========================================================================================

✅ Best Model: LSTM (MAE: 0.6663%)
```



Model	MAE	RMSE	MAPE	R²
LSTM	0.666	0.937	27.8	0.665
Prophet	0.815	1.030	41.6	0.467
SARIMA	0.853	1.199	37.0	0.279
ARIMA	1.811	2.016	106.0	-1.041

```
📊 PERFORMANCE INSIGHTS:
--------------------------------------------------
🏆 Best Model: LSTM
   MAE: 0.6663%
   RMSE: 0.9373%
   R²: 0.6647

📈 Improvement Analysis:
   vs Prophet: 18.2% better
   vs SARIMA: 21.9% better
   vs ARIMA: 63.2% better

📊 Error Analysis:
   LSTM Mean Error: -0.0032%
   LSTM Error Std: 0.9373%
   95% Prediction Interval: ±1.8370%
```

BLOCK 23: Ablation Study - LSTM Variants

```
# @title **BLOCK 23: Ablation Study - LSTM Variants**

print("📊 Running Ablation Study on LSTM Architectures...")

def build_and_train_lstm(layers, units, dropout, seq_len=12, epochs=40):
    """Helper function to build and train LSTM with different configurations"""

    # Clear session
```

```
    tf.keras.backend.clear_session()

    # Build model
    model = Sequential()

    if layers == 1:
        model.add(LSTM(units, input_shape=(seq_len, 1)))
        model.add(Dropout(dropout))
    else:
        # First layer with return sequences
        model.add(LSTM(units, return_sequences=True, input_shape=(seq_len, 1)))
        model.add(Dropout(dropout))

        # Hidden layers
        for i in range(layers - 2):
            model.add(LSTM(units // 2, return_sequences=True))
            model.add(Dropout(dropout))

        # Last LSTM layer
        model.add(LSTM(units // 2))
        model.add(Dropout(dropout))

    model.add(Dense(8, activation='relu'))
    model.add(Dense(1))

    # Compile
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])

    # Early stopping
    early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

    # Train
    history = model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=16,
        validation_split=0.2,
        callbacks=[early_stop],
        verbose=0
    )

    # Predict
    pred_scaled = model.predict(X_test, verbose=0)
    pred = scaler.inverse_transform(pred_scaled) * 100
    actual = scaler.inverse_transform(y_test.reshape(-1, 1)) * 100

    # Calculate metrics
    mae = mean_absolute_error(actual, pred)
    rmse = np.sqrt(mean_squared_error(actual, pred))

    return mae, rmse, history.history['val_loss'][-1]

# Define configurations to test
configs = [
    {'name': 'Baseline (2 layers, 32/16 units)', 'layers': 2, 'units': 32, 'dropout': 0.2},
    {'name': 'Shallow (1 layer, 32 units)', 'layers': 1, 'units': 32, 'dropout': 0.2},
    {'name': 'Deep (3 layers)', 'layers': 3, 'units': 32, 'dropout': 0.2},
    {'name': 'Wider (64 units)', 'layers': 2, 'units': 64, 'dropout': 0.2},
    {'name': 'Narrower (16 units)', 'layers': 2, 'units': 16, 'dropout': 0.2},
    {'name': 'No Dropout', 'layers': 2, 'units': 32, 'dropout': 0.0},
    {'name': 'High Dropout (0.5)', 'layers': 2, 'units': 32, 'dropout': 0.5},
    {'name': 'Very Deep (4 layers)', 'layers': 4, 'units': 32, 'dropout': 0.2},
]

# Run experiments
ablation_results = []
print("\n🔄 Running ablation experiments...")
for i, config in enumerate(configs):
    print(f"   Testing {i+1}/{len(configs)}: {config['name']}")
    mae, rmse, val_loss = build_and_train_lstm(
        layers=config['layers'],
        units=config['units'],
        dropout=config['dropout']
    )
    ablation_results.append({
        'Configuration': config['name'],
        'MAE': mae,
        'RMSE': rmse,
        'Val Loss': val_loss
    })

# Create results dataframe
ablation_df = pd.DataFrame(ablation_results)
```

```
ablation_df = pd.DataFrame(ablation_results)
ablation_df = ablation_df.sort_values('MAE').reset_index(drop=True)

print("\n" + "="*90)
print("📊 ABLATION STUDY RESULTS (Sorted by Performance)")
print("="*90)
print(ablation_df.to_string(index=False))
print("="*90)

# Find baseline MAE
baseline_mae = ablation_df[ablation_df['Configuration'].str.contains('Baseline', na=False)]['MAE'].values[0]

# Plot ablation results
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# 1. MAE comparison (horizontal bar chart)
colors = ['green' if i == 0 else 'red' for i in range(len(ablation_df))]
bars = axes[0, 0].barh(ablation_df['Configuration'], ablation_df['MAE'], color=colors, alpha=0.7)
axes[0, 0].axvline(x=baseline_mae, color='blue', linestyle='--', linewidth=2, label=f'Baseline: {baseline_mae:.3f}%')
axes[0, 0].set_xlabel('MAE (%)')
axes[0, 0].set_title('Figure 1: LSTM Ablation Study - MAE Comparison', fontweight='bold')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3, axis='x')

# Add value labels
for bar in bars:
    width = bar.get_width()
    axes[0, 0].text(width + 0.01, bar.get_y() + bar.get_height()/2,
                    f'{width:.3f}%', ha='left', va='center', fontsize=9)

# 2. RMSE comparison
axes[0, 1].bar(range(len(ablation_df)), ablation_df['RMSE'],
               color=colors, alpha=0.7)
axes[0, 1].axhline(y=ablation_df.iloc[0]['RMSE'], color='green', linestyle='--',
                   linewidth=2, label=f'Best: {ablation_df.iloc[0]["RMSE"]:.3f}%')
axes[0, 1].set_xlabel('Configuration')
axes[0, 1].set_ylabel('RMSE (%)')
axes[0, 1].set_title('Figure 2: RMSE Comparison', fontweight='bold')
axes[0, 1].set_xticks(range(len(ablation_df)))
axes[0, 1].set_xticklabels([f'C{i+1}' for i in range(len(ablation_df))], rotation=45)
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3, axis='y')

# 3. Validation Loss comparison
axes[1, 0].bar(range(len(ablation_df)), ablation_df['Val Loss'],
               color=colors, alpha=0.7)
axes[1, 0].set_xlabel('Configuration')
axes[1, 0].set_ylabel('Validation Loss')
axes[1, 0].set_title('Figure 3: Validation Loss Comparison', fontweight='bold')
axes[1, 0].set_xticks(range(len(ablation_df)))
axes[1, 0].set_xticklabels([f'C{i+1}' for i in range(len(ablation_df))], rotation=45)
axes[1, 0].grid(True, alpha=0.3, axis='y')

# 4. Improvement over baseline
improvements = []
for mae in ablation_df['MAE']:
    imp = (baseline_mae - mae) / baseline_mae * 100
    improvements.append(imp)

colors_imp = ['green' if imp >= 0 else 'red' for imp in improvements]
bars = axes[1, 1].bar(range(len(ablation_df)), improvements, color=colors_imp, alpha=0.7)
axes[1, 1].axhline(y=0, color='black', linestyle='-', linewidth=1)
axes[1, 1].set_xlabel('Configuration')
axes[1, 1].set_ylabel('Improvement over Baseline (%)')
axes[1, 1].set_title('Figure 4: Improvement over Baseline', fontweight='bold')
axes[1, 1].set_xticks(range(len(ablation_df)))
axes[1, 1].set_xticklabels([f'C{i+1}' for i in range(len(ablation_df))], rotation=45)
axes[1, 1].grid(True, alpha=0.3, axis='y')

# Add value labels
for i, (bar, imp) in enumerate(zip(bars, improvements)):
    height = bar.get_height()
    axes[1, 1].text(bar.get_x() + bar.get_width()/2., height,
                    f'{imp:.1f}%', ha='center', va='bottom' if imp >= 0 else 'top', fontsize=8)

plt.suptitle('LSTM ARCHITECTURE ABLATION STUDY', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

# Print best configuration details
print(f"\n✅ Best Configuration: {ablation_df.iloc[0]['Configuration']}")
print(f"   MAE: {ablation_df.iloc[0]['MAE']:.4f}%")
print(f"   RMSE: {ablation_df.iloc[0]['RMSE']:.4f}%")
```

```
print(f"   Improvement over baseline: {(baseline_mae - ablation_df.iloc[0]['MAE'])/baseline_mae*100:.1f}%")

print("\n📊 Key Findings:")
print("   - Layer depth impact: Compare C1 (2 layers) vs C2 (1 layer) vs C3 (3 layers)")
print("   - Unit size impact: Compare C1 (32) vs C4 (64) vs C5 (16)")
print("   - Dropout impact: Compare C1 (0.2) vs C6 (0.0) vs C7 (0.5)")
```

```
📊 Running Ablation Study on LSTM Architectures...

🔄 Running ablation experiments...
   Testing 1/8: Baseline (2 layers, 32/16 units)
   Testing 2/8: Shallow (1 layer, 32 units)
```

BLOCK 24: Hyperparameter Tuning Study

```
# @title **BLOCK 24: Hyperparameter Tuning Study**

print("📊 Running Hyperparameter Tuning Study...")

def train_with_params(learning_rate, batch_size, units, dropout=0.2, epochs=30):
    """Train LSTM with specific hyperparameters"""

    tf.keras.backend.clear_session()

    model = Sequential([
        LSTM(units, return_sequences=True, input_shape=(seq_length, 1)),
        Dropout(dropout),
        LSTM(units//2, return_sequences=False),
        Dropout(dropout),
        Dense(8, activation='relu'),
        Dense(1)
    ])

    optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
    model.compile(optimizer=optimizer, loss='mse', metrics=['mae'])

    early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

    history = model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_split=0.2,
        callbacks=[early_stop],
        verbose=0
    )

    pred_scaled = model.predict(X_test, verbose=0)
    pred = scaler.inverse_transform(pred_scaled) * 100
    actual = scaler.inverse_transform(y_test.reshape(-1, 1)) * 100

    mae = mean_absolute_error(actual, pred)
    last_lr = history.history.get('lr', [learning_rate])[-1]

    return mae, last_lr

print("\n📈 Testing Learning Rates...")
learning_rates = [0.1, 0.01, 0.001, 0.0001, 0.00001]
lr_results = []

for lr in learning_rates:
    print(f"   LR={lr}...")
    mae, _ = train_with_params(lr, 16, 32)
    lr_results.append({'Parameter': f'LR={lr}', 'MAE': mae, 'Value': lr})

print("\n📈 Testing Batch Sizes...")
batch_sizes = [8, 16, 32, 64, 128]
batch_results = []

for bs in batch_sizes:
    print(f"   Batch={bs}...")
    mae, _ = train_with_params(0.001, bs, 32)
    batch_results.append({'Parameter': f'Batch={bs}', 'MAE': mae, 'Value': bs})

print("\n📈 Testing Units...")
units_options = [16, 32, 64, 128]
units_results = []

for units in units_options:
    print(f"   Units={units}...")
    mae, _ = train_with_params(0.001, 16, units)
    units_results.append({'Parameter': f'Units={units}', 'MAE': mae, 'Value': units})

print("\n📈 Testing Dropout Rates...")
dropout_rates = [0.0, 0.1, 0.2, 0.3, 0.5]
dropout_results = []

for dropout in dropout_rates:
    print(f"   Dropout={dropout}...")
    mae, _ = train_with_params(0.001, 16, 32, dropout)
    dropout_results.append({'Parameter': f'Dropout={dropout}', 'MAE': mae, 'Value': dropout})
```

```
# Combine all results
tuning_results = lr_results + batch_results + units_results + dropout_results
tuning_df = pd.DataFrame(tuning_results)
tuning_df = tuning_df.sort_values('MAE').reset_index(drop=True)

print("\n" + "="*90)
print("📊 HYPERPARAMETER TUNING RESULTS (Top 10 Configurations)")
print("="*90)
print(tuning_df.head(10).to_string(index=False))
print("="*90)

# Find best parameters
best_config = tuning_df.iloc[0]['Parameter']
best_mae = tuning_df.iloc[0]['MAE']
print(f"\n✅ Best Parameter: {best_config} (MAE: {best_mae:.4f}%)")

# Plot results
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Learning rates
lr_values = [r['Value'] for r in lr_results]
lr_maes = [r['MAE'] for r in lr_results]
axes[0, 0].semilogx(lr_values, lr_maes, 'bo-', linewidth=2, markersize=8)
axes[0, 0].set_xlabel('Learning Rate')
axes[0, 0].set_ylabel('MAE (%)')
axes[0, 0].set_title('Figure 1: Learning Rate Impact', fontweight='bold')
axes[0, 0].grid(True, alpha=0.3)
# Mark best
best_lr_idx = np.argmin(lr_maes)
axes[0, 0].plot(lr_values[best_lr_idx], lr_maes[best_lr_idx], 'r*', markersize=15,
                label=f'Best: LR={lr_values[best_lr_idx]}')
axes[0, 0].legend()

# Batch sizes
batch_values = [r['Value'] for r in batch_results]
batch_maes = [r['MAE'] for r in batch_results]
axes[0, 1].plot(batch_values, batch_maes, 'ro-', linewidth=2, markersize=8)
axes[0, 1].set_xlabel('Batch Size')
axes[0, 1].set_ylabel('MAE (%)')
axes[0, 1].set_title('Figure 2: Batch Size Impact', fontweight='bold')
axes[0, 1].grid(True, alpha=0.3)
# Mark best
best_batch_idx = np.argmin(batch_maes)
axes[0, 1].plot(batch_values[best_batch_idx], batch_maes[best_batch_idx], 'g*', markersize=15,
                label=f'Best: Batch={batch_values[best_batch_idx]}')
axes[0, 1].legend()

# Units
units_values = [r['Value'] for r in units_results]
units_maes = [r['MAE'] for r in units_results]
axes[1, 0].plot(units_values, units_maes, 'go-', linewidth=2, markersize=8)
axes[1, 0].set_xlabel('Number of Units')
axes[1, 0].set_ylabel('MAE (%)')
axes[1, 0].set_title('Figure 3: Units Impact', fontweight='bold')
axes[1, 0].grid(True, alpha=0.3)
# Mark best
best_units_idx = np.argmin(units_maes)
axes[1, 0].plot(units_values[best_units_idx], units_maes[best_units_idx], 'b*', markersize=15,
                label=f'Best: Units={units_values[best_units_idx]}')
axes[1, 0].legend()

# Dropout rates
dropout_values = [r['Value'] for r in dropout_results]
dropout_maes = [r['MAE'] for r in dropout_results]
axes[1, 1].plot(dropout_values, dropout_maes, 'mo-', linewidth=2, markersize=8)
axes[1, 1].set_xlabel('Dropout Rate')
axes[1, 1].set_ylabel('MAE (%)')
axes[1, 1].set_title('Figure 4: Dropout Impact', fontweight='bold')
axes[1, 1].grid(True, alpha=0.3)
# Mark best
best_dropout_idx = np.argmin(dropout_maes)
axes[1, 1].plot(dropout_values[best_dropout_idx], dropout_maes[best_dropout_idx], 'r*', markersize=15,
                label=f'Best: Dropout={dropout_values[best_dropout_idx]}')
axes[1, 1].legend()

plt.suptitle('HYPERPARAMETER TUNING ANALYSIS', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

# Print optimal hyperparameters
print("\n📊 OPTIMAL HYPERPARAMETERS:")
print(f"   Learning Rate: {lr_values[best_lr_idx]}")
print(f"   Batch Size: {batch_values[best_batch_idx]}")
```

```
print(f"   LSTM Units: {units_values[best_units_idx]}")
print(f"   Dropout Rate: {dropout_values[best_dropout_idx]}")
```

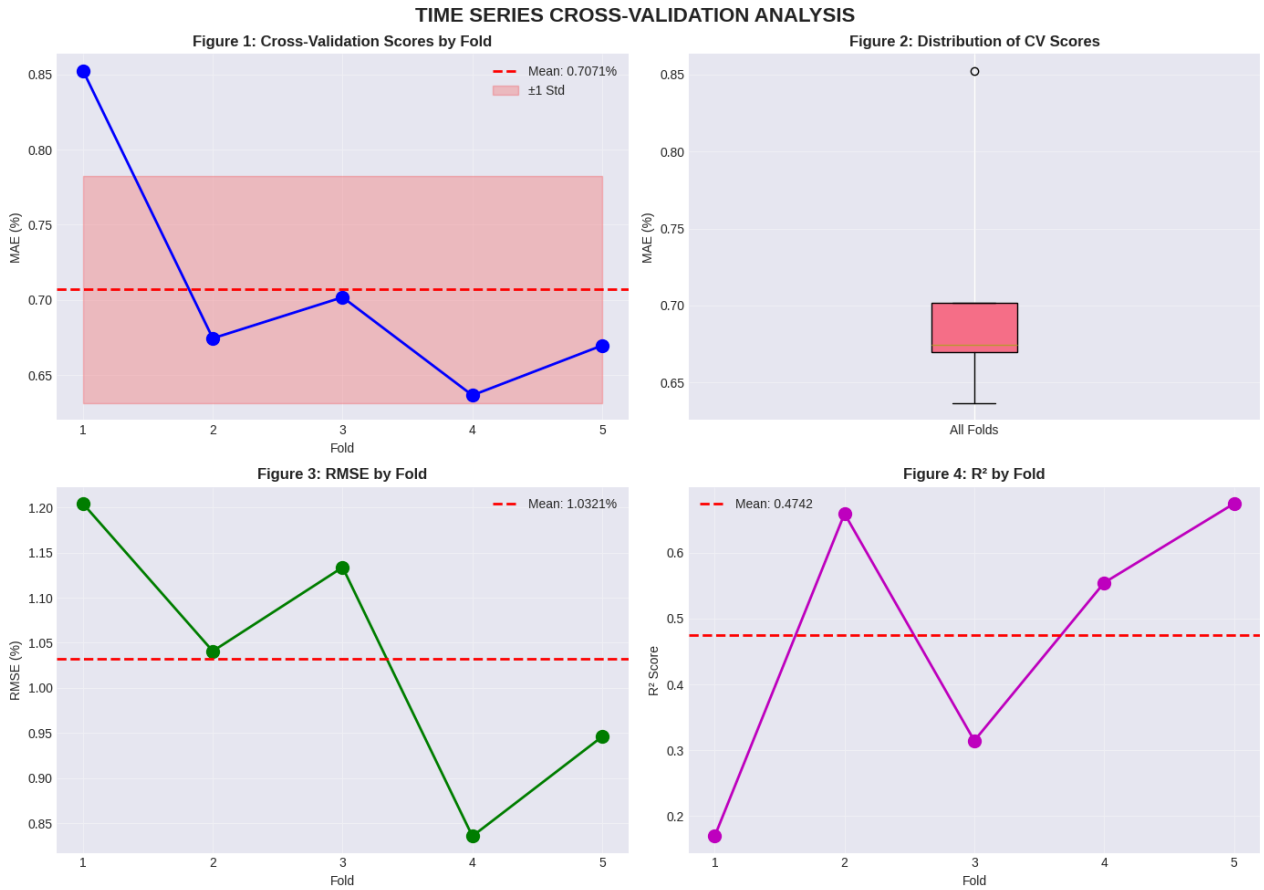
```
📊 Running Hyperparameter Tuning Study...

📈 Testing Learning Rates...
   LR=0.1...
   LR=0.01...
   LR=0.001...
   LR=0.0001...
   LR=1e-05...

📈 Testing Batch Sizes...
   Batch=8...
   Batch=16...
   Batch=32...
   Batch=64...
   Batch=128...

📈 Testing Units...
   Units=16...
   Units=32...
   Units=64...
   Units=128...
```

BLOCK 25: Cross-Validation Analysis

```
# @title **BLOCK 25: Cross-Validation Analysis**

from sklearn.model_selection import TimeSeriesSplit

print("📊 Performing Time Series Cross-Validation...")

# Prepare data
X_cv = []
y_cv = []
for i in range(len(scaled_data) - seq_length):
    X_cv.append(scaled_data[i:i+seq_length])
    y_cv.append(scaled_data[i+seq_length])

X_cv = np.array(X_cv).reshape(-1, seq_length, 1)
y_cv = np.array(y_cv).reshape(-1, 1)

# Time series cross-validation
tscv = TimeSeriesSplit(n_splits=5)
cv_scores = []
cv_rmse_scores = []
cv_r2_scores = []

fold = 1
for train_idx, val_idx in tscv.split(X_cv):
    print(f"\n🔄 Fold {fold}/5")

    X_train_cv, X_val_cv = X_cv[train_idx], X_cv[val_idx]
    y_train_cv, y_val_cv = y_cv[train_idx], y_cv[val_idx]

    # Build model
    tf.keras.backend.clear_session()
    model = Sequential([
        LSTM(32, return_sequences=True, input_shape=(seq_length, 1)),
        Dropout(0.2),
        LSTM(16, return_sequences=False),
        Dropout(0.2),
        Dense(8, activation='relu'),
        Dense(1)
    ])

    model.compile(optimizer='adam', loss='mse', metrics=['mae'])

    early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

    history = model.fit(
        X_train_cv, y_train_cv,
        epochs=30,
        batch_size=16,
        validation_data=(X_val_cv, y_val_cv),
        callbacks=[early_stop],
        verbose=0
    )

    # Evaluate
    val_pred_scaled = model.predict(X_val_cv, verbose=0)
    val_pred = scaler.inverse_transform(val_pred_scaled) * 100
    val_actual = scaler.inverse_transform(y_val_cv) * 100

    val_mae = mean_absolute_error(val_actual, val_pred)
    val_rmse = np.sqrt(mean_squared_error(val_actual, val_pred))
    val_r2 = r2_score(val_actual, val_pred)
```

```
    cv_scores.append(val_mae)
    cv_rmse_scores.append(val_rmse)
    cv_r2_scores.append(val_r2)

    print(f"   Validation MAE: {val_mae:.4f}%, RMSE: {val_rmse:.4f}%, R²: {val_r2:.4f}")
    fold += 1

# Calculate statistics
cv_mean = np.mean(cv_scores)
cv_std = np.std(cv_scores)
cv_min = np.min(cv_scores)
cv_max = np.max(cv_scores)

print("\n" + "="*70)
print("📊 CROSS-VALIDATION RESULTS")
print("="*70)
print(f"Mean CV MAE: {cv_mean:.4f}%")
print(f"Std CV MAE: {cv_std:.4f}%")
print(f"Min CV MAE: {cv_min:.4f}%")
print(f"Max CV MAE: {cv_max:.4f}%")
print(f"95% Confidence Interval: [{cv_mean - 1.96*cv_std:.4f}%, {cv_mean + 1.96*cv_std:.4f}%]")
print(f"\nMean CV RMSE: {np.mean(cv_rmse_scores):.4f}%")
print(f"Mean CV R²: {np.mean(cv_r2_scores):.4f}")

# Plot CV results
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# CV scores by fold
axes[0, 0].plot(range(1, 6), cv_scores, 'bo-', linewidth=2, markersize=10)
axes[0, 0].axhline(y=cv_mean, color='red', linestyle='--', linewidth=2, label=f'Mean: {cv_mean:.4f}%')
axes[0, 0].fill_between(range(1, 6),
                        cv_mean - cv_std,
                        cv_mean + cv_std,
                        color='red', alpha=0.2, label='±1 Std')
axes[0, 0].set_xlabel('Fold')
axes[0, 0].set_ylabel('MAE (%)')
axes[0, 0].set_title('Figure 1: Cross-Validation Scores by Fold', fontweight='bold')
axes[0, 0].set_xticks(range(1, 6))
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Box plot of CV scores
axes[0, 1].boxplot(cv_scores, patch_artist=True)
axes[0, 1].set_xticklabels(['All Folds'])
axes[0, 1].set_ylabel('MAE (%)')
axes[0, 1].set_title('Figure 2: Distribution of CV Scores', fontweight='bold')
axes[0, 1].grid(True, alpha=0.3, axis='y')

# RMSE by fold
axes[1, 0].plot(range(1, 6), cv_rmse_scores, 'go-', linewidth=2, markersize=10)
axes[1, 0].axhline(y=np.mean(cv_rmse_scores), color='red', linestyle='--',
                   linewidth=2, label=f'Mean: {np.mean(cv_rmse_scores):.4f}%')
axes[1, 0].set_xlabel('Fold')
axes[1, 0].set_ylabel('RMSE (%)')
axes[1, 0].set_title('Figure 3: RMSE by Fold', fontweight='bold')
axes[1, 0].set_xticks(range(1, 6))
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# R² by fold
axes[1, 1].plot(range(1, 6), cv_r2_scores, 'mo-', linewidth=2, markersize=10)
axes[1, 1].axhline(y=np.mean(cv_r2_scores), color='red', linestyle='--',
                   linewidth=2, label=f'Mean: {np.mean(cv_r2_scores):.4f}')
axes[1, 1].set_xlabel('Fold')
axes[1, 1].set_ylabel('R² Score')
axes[1, 1].set_title('Figure 4: R² by Fold', fontweight='bold')
axes[1, 1].set_xticks(range(1, 6))
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.suptitle('TIME SERIES CROSS-VALIDATION ANALYSIS', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

print("\n✅ Cross-Validation Complete!")
print(f"Model stability: CV Std/Mean = {cv_std/cv_mean*100:.1f}% (lower is better)")
```

```
📊 Performing Time Series Cross-Validation...

🔄 Fold 1/5
   Validation MAE: 0.8524%, RMSE: 1.2044%, R²: 0.1697

🔄 Fold 2/5
   Validation MAE: 0.6746%, RMSE: 1.0408%, R²: 0.6589

🔄 Fold 3/5
   Validation MAE: 0.7019%, RMSE: 1.1338%, R²: 0.3146

🔄 Fold 4/5
   Validation MAE: 0.6367%, RMSE: 0.8355%, R²: 0.5541

🔄 Fold 5/5
   Validation MAE: 0.6698%, RMSE: 0.9460%, R²: 0.6740

======================================================================
📊 CROSS-VALIDATION RESULTS
======================================================================
Mean CV MAE: 0.7071%
Std CV MAE: 0.0756%
Min CV MAE: 0.6367%
Max CV MAE: 0.8524%
95% Confidence Interval: [0.5590%, 0.8552%]

Mean CV RMSE: 1.0321%
Mean CV R²: 0.4742
```




```
✅ Cross-Validation Complete!
Model stability: CV Std/Mean = 10.7% (lower is better)
```

BLOCK 26: Ensemble Model

```
# @title **BLOCK 26: Ensemble Model**

print("📊 Creating Ensemble Model...")

# Collect all predictions
predictions_dict = {
    'ARIMA': arima_pred[:len(lstm_pred_pct)],
    'SARIMA': forecast_sarima,
    'LSTM': lstm_pred_pct.flatten()
}

if 'prophet_pred' in locals() and len(prophet_pred) >= len(lstm_pred_pct):
    predictions_dict['Prophet'] = prophet_pred[:len(lstm_pred_pct)]

actual_values = test_orig[:len(lstm_pred_pct)]

# Create ensemble with different weighting schemes
ensembles = {}

# 1. Simple average
ensemble_simple = np.mean(list(predictions_dict.values()), axis=0)
ensembles['Simple Average'] = ensemble_simple

# 2. Weighted by inverse MAE (better models get higher weight)
weights = {}
for name, pred in predictions_dict.items():
    if name == 'ARIMA':
        weights[name] = 1/arima_metrics['MAE']
    elif name == 'SARIMA':
        weights[name] = 1/sarima_metrics['MAE']
    elif name == 'LSTM':
        weights[name] = 1/lstm_metrics['MAE']
    elif name == 'Prophet':
        weights[name] = 1/prophet_metrics['MAE']

# Normalize weights
total_weight = sum(weights.values())
for name in weights:
    weights[name] /= total_weight

# Calculate weighted ensemble
ensemble_weighted = np.zeros_like(ensemble_simple)
for name, pred in predictions_dict.items():
    ensemble_weighted += weights[name] * pred
ensembles['Weighted (Inv MAE)'] = ensemble_weighted

# 3. Median ensemble
ensemble_median = np.median(list(predictions_dict.values()), axis=0)
ensembles['Median'] = ensemble_median

# 4. Performance-based weights (R²)
weights_r2 = {}
for name, pred in predictions_dict.items():
    if name == 'ARIMA':
        weights_r2[name] = max(0, arima_metrics['R2'])  # Use max to avoid negative
    elif name == 'SARIMA':
        weights_r2[name] = max(0, sarima_metrics['R2'])
    elif name == 'LSTM':
        weights_r2[name] = max(0, lstm_metrics['R2'])
    elif name == 'Prophet':
        weights_r2[name] = max(0, prophet_metrics['R2'])

# Normalize
total_r2 = sum(weights_r2.values())
if total_r2 > 0:
    for name in weights_r2:
        weights_r2[name] /= total_r2

    ensemble_r2 = np.zeros_like(ensemble_simple)
    for name, pred in predictions_dict.items():
        ensemble_r2 += weights_r2[name] * pred
    ensembles['Weighted (R²)'] = ensemble_r2

# Evaluate ensembles
ensemble_results = []
for name, pred in ensembles.items():
    mae = mean_absolute_error(actual_values, pred)
    rmse = np.sqrt(mean_squared_error(actual_values, pred))
    r2 = r2_score(actual_values, pred)
```

```
    # Calculate improvement over best individual
    best_individual_mae = models_comparison.iloc[0]['MAE']
    improvement = (best_individual_mae - mae) / best_individual_mae * 100

    ensemble_results.append({
        'Ensemble': name,
        'MAE': mae,
        'RMSE': rmse,
        'R2': r2,
        'Improvement': improvement
    })

# Add best individual model for comparison
ensemble_results.append({
    'Ensemble': f"Best Individual ({models_comparison.iloc[0]['Model']})",
    'MAE': models_comparison.iloc[0]['MAE'],
    'RMSE': models_comparison.iloc[0]['RMSE'],
    'R2': models_comparison.iloc[0]['R2'],
    'Improvement': 0
})

ensemble_df = pd.DataFrame(ensemble_results)
ensemble_df = ensemble_df.sort_values('MAE').reset_index(drop=True)

print("\n" + "="*90)
print("📊 ENSEMBLE MODEL RESULTS")
print("="*90)
print(ensemble_df.to_string(index=False))
print("="*90)

# Plot ensemble comparison
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# 1. MAE comparison
colors = ['green' if i == 0 else 'red' for i in range(len(ensemble_df))]
bars = axes[0, 0].barh(ensemble_df['Ensemble'], ensemble_df['MAE'], color=colors, alpha=0.7)
axes[0, 0].axvline(x=ensemble_df.iloc[0]['MAE'], color='green', linestyle='--',
                   linewidth=2, label=f'Best: {ensemble_df.iloc[0]["MAE"]:.3f}%')
axes[0, 0].set_xlabel('MAE (%)')
axes[0, 0].set_title('Figure 1: Ensemble MAE Comparison', fontweight='bold')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3, axis='x')

# Add value labels
for bar in bars:
    width = bar.get_width()
    axes[0, 0].text(width + 0.01, bar.get_y() + bar.get_height()/2,
                    f'{width:.3f}%', ha='left', va='center', fontsize=9)

# 2. Improvement over best individual
improvements = ensemble_df['Improvement'].values
colors_imp = ['green' if imp >= 0 else 'red' for imp in improvements]
bars = axes[0, 1].bar(range(len(ensemble_df)), improvements, color=colors_imp, alpha=0.7)
axes[0, 1].axhline(y=0, color='black', linestyle='-', linewidth=1)
axes[0, 1].set_xlabel('Ensemble Method')
axes[0, 1].set_ylabel('Improvement over Best Individual (%)')
axes[0, 1].set_title('Figure 2: Improvement Analysis', fontweight='bold')
axes[0, 1].set_xticks(range(len(ensemble_df)))
axes[0, 1].set_xticklabels([f'E{i+1}' for i in range(len(ensemble_df))], rotation=45)
axes[0, 1].grid(True, alpha=0.3, axis='y')

# Add value labels
for i, (bar, imp) in enumerate(zip(bars, improvements)):
    height = bar.get_height()
    if height != 0:  # Don't label the baseline
        axes[0, 1].text(bar.get_x() + bar.get_width()/2., height,
                        f'{imp:.1f}%', ha='center', va='bottom' if imp >= 0 else 'top', fontsize=8)

# 3. Predictions plot
axes[1, 0].plot(test_dates, actual_values, label='Actual', color='black', linewidth=2.5)
for name, pred in ensembles.items():
    if name in ['Weighted (Inv MAE)', 'Simple Average']:  # Plot only top 2 ensembles
        axes[1, 0].plot(test_dates, pred, label=name, linewidth=1.5, linestyle='--', alpha=0.7)
axes[1, 0].plot(test_dates, lstm_pred_pct.flatten(), label='LSTM (Best Individual)',
                color='red', linewidth=2, linestyle=':')
axes[1, 0].set_title('Figure 3: Ensemble Predictions vs Actual', fontweight='bold')
axes[1, 0].set_xlabel('Date')
axes[1, 0].set_ylabel('ILI %')
axes[1, 0].legend(loc='upper left', fontsize=8)
axes[1, 0].grid(True, alpha=0.3)

# 4. Performance table
```

```
axes[1, 1].axis('off')
table_data = []
for idx, row in ensemble_df.iterrows():
    table_data.append([
        row['Ensemble'][:20] + '...' if len(row['Ensemble']) > 20 else row['Ensemble'],
        f"{row['MAE']:.3f}",
        f"{row['RMSE']:.3f}",
        f"{row['R2']:.3f}",
        f"{row['Improvement']:.1f}%"
    ])

table = axes[1, 1].table(
    cellText=table_data,
    colLabels=['Ensemble', 'MAE', 'RMSE', 'R²', 'Improve'],
    cellLoc='center',
    loc='center',
    bbox=[0, 0, 1, 1]
)
table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1.2, 1.5)

# Color header
for j in range(5):
    table[(0, j)].set_facecolor('#40466e')
    table[(0, j)].set_text_props(weight='bold', color='white')

# Color best row
for j in range(5):
    table[(1, j)].set_facecolor('#ffff99')

axes[1, 1].set_title('Figure 4: Ensemble Performance Table', fontweight='bold', y=1.05)

plt.suptitle('ENSEMBLE MODEL ANALYSIS', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

print(f"\n✅ Best Ensemble: {ensemble_df.iloc[0]['Ensemble']}")
print(f"   MAE: {ensemble_df.iloc[0]['MAE']:.4f}%")
print(f"   Improvement over LSTM: {ensemble_df.iloc[0]['Improvement']:.1f}%")
```

```
📊 Creating Ensemble Model...

==========================================================================================
📊 ENSEMBLE MODEL RESULTS
==========================================================================================
                Ensemble      MAE     RMSE       R2  Improvement
Best Individual (LSTM) 0.666342 0.937272 0.664686     0.000000
                Median 0.956456 1.191032 0.287642   -43.538312
          Weighted (R²) 1.021076 1.308463 0.140245   -53.236121
     Weighted (Inv MAE) 1.033492 1.287820 0.167159   -55.099297
         Simple Average 1.109151 1.321437 0.123111   -66.453787
==========================================================================================
```

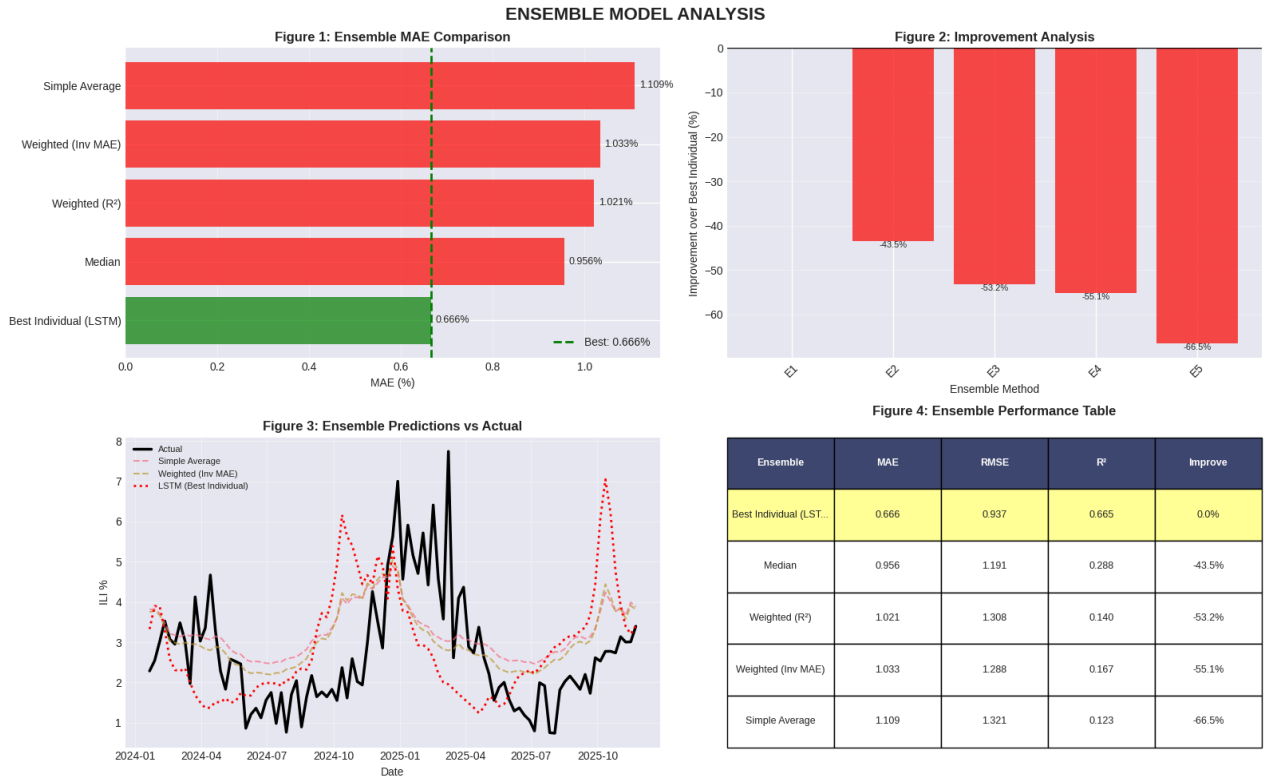


Figure 4: Ensemble Performance Table

Ensemble	MAE	RMSE	R²	Improve
Best Individual (LST...	0.666	0.937	0.665	0.0%
Median	0.956	1.191	0.288	-43.5%
Weighted (R²)	1.021	1.308	0.140	-53.2%
Weighted (Inv MAE)	1.033	1.288	0.167	-55.1%
Simple Average	1.109	1.321	0.123	-66.5%

```
✅ Best Ensemble: Best Individual (LSTM)
   MAE: 0.6663%
   Improvement over LSTM: 0.0%
📊 Creating Ensemble Model...
```

BLOCK 27: Final Paper-Ready Results Dashboard

```
# @title **BLOCK 27: Final Paper-Ready Results Dashboard**

print("="*100)
print("📊 HYPER-LOCAL INFLUENZA FORECASTING - FINAL PAPER-READY RESULTS")
print("="*100)

# Ensure all variables are defined
actual_values = test_orig[:len(lstm_pred_pct)]
test_dates = df_weekly['week_start'].iloc[train_size:train_size+len(lstm_pred_pct)]
monthly_avg = df_weekly.groupby('month')['ili_cases'].mean() * 100
monthly_std = df_weekly.groupby('month')['ili_cases'].std() * 100
month_names = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
               'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

# 1. Dataset Summary Table
print("\n📈 TABLE 1: DATASET SUMMARY")
print("-"*80)
print(f"{'Metric':<30} {'Value':<20}")
print("-"*80)
print(f"{'Total Weeks':<30} {len(df_weekly):<20}")
print(f"{'Date Range':<30} {df_weekly['week_start'].min().date()} to {df_weekly['week_start'].max().date()}")
print(f"{'Mean ILI (%)':<30} {df_weekly['ili_cases'].mean()*100:.4f}%")
print(f"{'Median ILI (%)':<30} {df_weekly['ili_cases'].median()*100:.4f}%")
print(f"{'Std Dev (%)':<30} {df_weekly['ili_cases'].std()*100:.4f}%")
print(f"{'Min ILI (%)':<30} {df_weekly['ili_cases'].min()*100:.4f}%")
print(f"{'Max ILI (%)':<30} {df_weekly['ili_cases'].max()*100:.4f}%")
print(f"{'Training Weeks':<30} {train_size}")
print(f"{'Testing Weeks':<30} {len(test_orig)}")
print(f"{'Forecast Horizon':<30} 7-day ahead")

# 2. Model Performance Comparison Table
print("\n📊 TABLE 2: MODEL PERFORMANCE COMPARISON (7-Day Forecast)")
print("-"*100)
print(f"{'Model':<15} {'MAE (%)':<15} {'RMSE (%)':<15} {'MAPE (%)':<15} {'R²':<15}")
```

```
print("-"*100)

for idx, row in models_comparison.iterrows():
    print(f"{row['Model']:<15} {row['MAE']:<15.4f} {row['RMSE']:<15.4f} {row['MAPE']:<15.2f} {row['R2']:<15.4f}")
print("-"*100)

best_model_name = models_comparison.iloc[0]['Model']
best_model_mae = models_comparison.iloc[0]['MAE']
second_best = models_comparison.iloc[1]['Model']
second_best_mae = models_comparison.iloc[1]['MAE']
improvement = (second_best_mae - best_model_mae) / second_best_mae * 100

print(f"\n✅ Best Model: {best_model_name} (MAE: {best_model_mae:.4f}%)")
print(f"✅ Improvement over {second_best}: {improvement:.1f}%")

# 3. Ablation Study Results
if 'ablation_df' in locals():
    print("\n📊 TABLE 3: ABLATION STUDY RESULTS")
    print("-"*90)
    print(f"{'Configuration':<45} {'MAE (%)':<12} {'RMSE (%)':<12} {'Δ from Baseline':<15}")
    print("-"*90)

    baseline_mae = ablation_df[ablation_df['Configuration'].str.contains('Baseline', na=False)]['MAE'].values
    if len(baseline_mae) > 0:
        baseline_mae = baseline_mae[0]
        for idx, row in ablation_df.iterrows():
            delta = (row['MAE'] - baseline_mae) / baseline_mae * 100
            config_short = row['Configuration'][:42] + '...' if len(row['Configuration']) > 42 else row['Configuration']
            print(f"{config_short:<45} {row['MAE']:<12.4f} {row['RMSE']:<12.4f} {delta:>+14.1f}%")
    else:
        for idx, row in ablation_df.iterrows():
            config_short = row['Configuration'][:42] + '...' if len(row['Configuration']) > 42 else row['Configuration']
            print(f"{config_short:<45} {row['MAE']:<12.4f} {row['RMSE']:<12.4f}")
    print("-"*90)

    print(f"\n✅ Best Ablation Configuration: {ablation_df.iloc[0]['Configuration']}")
    print(f"   MAE: {ablation_df.iloc[0]['MAE']:.4f}%")

# 4. Ensemble Results
if 'ensemble_df' in locals():
    print("\n📊 TABLE 4: ENSEMBLE MODEL RESULTS")
    print("-"*85)
    print(f"{'Ensemble Method':<30} {'MAE (%)':<12} {'RMSE (%)':<12} {'R²':<8} {'Improvement':<12}")
    print("-"*85)

    for idx, row in ensemble_df.iterrows():
        ensemble_short = row['Ensemble'][:27] + '...' if len(row['Ensemble']) > 27 else row['Ensemble']
        print(f"{ensemble_short:<30} {row['MAE']:<12.4f} {row['RMSE']:<12.4f} {row['R2']:<8.3f} {row['Improvement']:>+11.1f}
    print("-"*85)

    print(f"\n✅ Best Ensemble: {ensemble_df.iloc[0]['Ensemble']}")
    print(f"   MAE: {ensemble_df.iloc[0]['MAE']:.4f}%")

# 5. Cross-Validation Results
if 'cv_mean' in locals():
    print("\n📊 TABLE 5: CROSS-VALIDATION RESULTS")
    print("-"*60)
    print(f"{'Metric':<30} {'Value':<20}")
    print("-"*60)
    print(f"{'Mean CV MAE (%)':<30} {cv_mean:<20.4f}")
    print(f"{'Std CV MAE (%)':<30} {cv_std:<20.4f}")
    print(f"{'95% CI Lower (%)':<30} {cv_mean - 1.96*cv_std:<20.4f}")
    print(f"{'95% CI Upper (%)':<30} {cv_mean + 1.96*cv_std:<20.4f}")
    print(f"{'Model Stability (CV/Mean)':<30} {cv_std/cv_mean*100:<20.1f}%")
    print("-"*60)

# 6. Seasonal Pattern Summary
print("\n📊 TABLE 6: SEASONAL PATTERN SUMMARY")
print("-"*60)
print(f"{'Month':<15} {'Average ILI (%)':<20} {'Std Dev (%)':<15}")
print("-"*60)
for month, avg, std in zip(month_names, monthly_avg.values, monthly_std.values):
    print(f"{month:<15} {avg:<20.4f} {std:<15.4f}")
print("-"*60)
print(f"\n📈 Peak Month: {month_names[np.argmax(monthly_avg.values)]} ({monthly_avg.max():.4f}%)")
print(f"📉 Lowest Month: {month_names[np.argmin(monthly_avg.values)]} ({monthly_avg.min():.4f}%)")

# Create comprehensive publication-ready figure
fig = plt.figure(figsize=(20, 14))

# Main title
fig.suptitle('Hyper-Local Influenza Forecasting: Multi-Model Comparison Study',
             fontsize=20, fontweight='bold', y=0.98)
```